

```
// ===== FILE: utils/chat-compress.js =====
/**
 * 聊天服务 ---- 压缩域:
 * 把上文交给 LLM 压缩为概要 (手动触发 / 每轮回复后自动检查)
 */

import { getChatRows, getConversationCompression, setConversationCompression } from './storage.js'
import { chatCompletion } from './llm.js'
import { addLog } from './log.js'
import { getConversationSettings } from './chat-settings.js'

const COMPRESS_KEEP_TAIL = 10 // 压缩时保留的最近完整消息条数
const COMPRESS_MIN_NEW = 4 // 每次压缩至少新增这么多条才执行
const COMPRESS_CHUNK = 80 // 单次压缩请求处理的最大消息条数 (超出分批多次调用)
const COMPRESS_SYSTEM = `你是对话压缩助手。把对话压缩成一份精炼概要，用于替代原文作为后续上下文。
要求：保留关键事实、用户偏好、已做的决定、约定和正在进行的事；保留重要人物/时间/地点信息；
用中文，150 字以内，只输出概要本身，不要解释、不要分段标题。`

let _compressing = false

/**
 * 把上文交给 LLM 压缩为概要 (手动或自动触发)。
 * 压缩范围：上次压缩进度之后、保留尾部之前的消息；超过单批上限时分批调用。
 * @param {boolean} force 手动触发传 true，忽略间隔/新增量限制
 * @returns {Promise<boolean>} 是否实际执行了压缩
 */
export async function compressContext(force = false) {
  if (_compressing) return false
  const rows = getChatRows()
  const keepTail = COMPRESS_KEEP_TAIL
  if (rows.length <= keepTail + COMPRESS_MIN_NEW) return false

  const { summary: prev, compressedUntil } = getConversationCompression()
  const since = Math.min(compressedUntil, rows.length - keepTail)
  const cut = Math.max(since, rows.length - keepTail)
  if (!force && cut - since < COMPRESS_MIN_NEW) return false
  if (cut <= since) return false

  const s = getConversationSettings()
  if (!s.apiKey || !s.baseUrl || !s.model) return false

  _compressing = true
  try {
    let merged = prev
    let cursor = since
    // 分批压缩，后一批把前一批的概要合并进来，避免单次请求过大
    while (cursor < cut) {
      const end = Math.min(cursor + COMPRESS_CHUNK, cut)
      const batch = rows.slice(cursor, end)

      const text = [
```

```

merged ? `已有的对话概要: \n${merged}` : '',
' 以下是对话记录 (越靠后越新): ',
batch.map((r) => `${r.role === 'user' ? ' 用户' : ' 你': ${r.content}}`).join('\n'),
' 请把已有概要与新内容合并, 输出一份完整的精炼概要。'
].filter(Boolean).join('\n\n')
const resp = await chatCompletion({
  baseUrl: s.baseUrl,
  apiKey: s.apiKey,
  model: s.model,
  messages: [
    { role: 'system', content: COMPRESS_SYSTEM },
    { role: 'user', content: text }
  ],
  temperature: 0.3,
  maxTokens: 512,
  reasoningEffort: 'none' // 压缩任务简单, 固定关闭思考, 避免被思考 token 占满
})
const out = (resp.text || '').trim()
if (!out) return false
merged = out
cursor = end
}
setConversationCompression(merged, cut)
addLog(
  'info',
  ' 压缩上文',
  `压缩 ${cut - since} 条消息为概要 (保留尾部 ${keepTail} 条) \n--- 压缩后上文 ---\n${merged}`
)
return true
} catch (e) {
  addLog('err', ' 压缩上文失败', e.message)
  throw e
} finally {
  _compressing = false
}
}

/**
 * 自动压缩检查: 距上次压缩新增消息达到间隔阈值则压缩。
 * 在每次对话回复落库后调用 (异步、失败静默, 不阻塞回复)。
 */
export async function maybeCompress() {
  if (_compressing) return false
  const interval = getConversationSettings().compressInterval
  if (!interval) return false
  const { compressedUntil } = getConversationCompression()
  if (getChatRows().length - compressedUntil < interval) return false
  addLog('info', ' 自动压缩触发', `间隔 ${interval} 条`)
  return compressContext()
}

```

```
}

// ===== FILE: utils/chat-conversations.js =====
/**
 * 聊天服务 ---- 会话域:
 * 多会话管理 (新建/切换/删除/复制) + 当前会话历史展示/清空 + 最近请求缓存同步
 */

import {
  getChatRows,
  clearChat,
  getConversations,
  getActiveConversationId,
  createConversation,
  switchConversation,
  deleteConversation,
  duplicateConversationToNew,
  duplicateMemoriesToNew,
  setConversationSettingsRaw
} from './storage.js'
import { memoryStore, lastRequest } from './chat-state.js'
import { getConversationSettings, normalizeSettings } from './chat-settings.js'
import { addLog } from './log.js'

/** 剔除消息文本行首的 Scene/Memory 标记行 (修复旧数据: 早期落库过含标记的完整回复) */
function stripMetaLines(content) {
  const clean = String(content || '')
    .split('\n')
    .filter((l) => !/^s*(Scene|Memory)s*[: ]/i.test(l))
    .join('\n')
    .trim()
  return clean || '...'
}

/** 聊天页历史 (返回副本, 剔除遗留的 Scene/Memory 标记行后展示) */
export function getHistoryForUI() {
  return getChatRows().map((r) => ({ ...r, content: stripMetaLines(r.content) }))
}

/** 清空对话 */
export function clearConversation() {
  clearChat()
  syncLastRequest()
  addLog('info', ' 清空当前对话')
}

/** 同步请求缓存到当前会话的最后一用户消息 (切会话/清空后调用, 避免跨会话重发错内容) */
export function syncLastRequest() {
  const rows = getChatRows()
```

```
    for (let i = rows.length - 1; i >= 0; i--) {
      if (rows[i].role === 'user') {
        lastRequest.value = rows[i].content
        return
      }
    }
    lastRequest.value = ''
  }

  // ----- 会话管理 -----

  /** 会话列表（供历史对话弹窗展示） */
  export function listConversations() {
    return getConversations()
  }

  /** 当前会话 id */
  export function activeConversationId() {
    return getActiveConversationId()
  }

  /** 开始新对话：当前对话非空则归档并新建，否则复用并重置当前空对话；新对话复制当前设置（每个对话独享一份设置） */
  export function startNewConversation() {
    memoryStore.resetState()
    // 记录创建前的当前会话生效设置（含无快照会话的全局回退值），供新对话复制
    const currentSettings = getConversationSettings()
    if (getChatRows().length) {
      createConversation()
      addLog('info', ' 开始新对话')
    } else {
      clearChat()
      addLog('info', ' 重置空对话')
    }
    setConversationSettingsRaw(normalizeSettings(currentSettings))
    syncLastRequest()
  }

  /** 切换到指定会话（记忆与人格随会话切换，重置召回状态） */
  export function openConversation(id) {
    const ok = switchConversation(id)
    if (ok) {
      memoryStore.resetState()
      syncLastRequest()
      addLog('info', ' 切换会话', id)
    }
    return ok
  }

  /** 删除指定会话 */
```

```
export function removeConversation(id) {
  const ok = deleteConversation(id)
  if (ok) {
    memoryStore.resetState()
    syncLastRequest()
    addLog('info', ' 删除会话', id)
  }
  return ok
}

/** 复制当前会话（消息 + 记忆 + 人格）到新会话并切换 */
export function copyConversationToNew() {
  duplicateConversationToNew()
  memoryStore.resetState()
  syncLastRequest()
  addLog('info', ' 复制对话到新会话')
}

/** 复制当前会话的记忆到新会话并切换 */
export function copyMemoriesToNew() {
  duplicateMemoriesToNew()
  memoryStore.resetState()
  syncLastRequest()
  addLog('info', ' 复制记忆到新会话')
}

// ===== FILE: utils/chat-proactive.js =====
/**
 * 聊天服务 ---- 拟真聊天域：
 * 前台定时器调度（随机时间节点主动发消息）+ 回前台补发（catch-up）+ 主动消息发送链路。
 *
 * 生效条件（当前会话）：proactiveEnabled 开启 且 timeMode=real（现实时间）。
 * 低后台占用设计：单条链式 setTimeout（非轮询，空闲零唤醒），退后台 ** 不清定时器 **----
 * 单条 pending timeout 后台零 CPU；H5 隐藏标签页会被浏览器节流（约 1 次/分钟，够用）；
 * App 后台 JS 挂起时定时器不触发，回前台由 catchUpProactive 检测到期补发。
 * ** 进程结束后重开（关页/杀进程）也不丢触发 **：到期时刻持久化到 storage，
 * 重开时 catchUpProactive 据此判断“关闭期间已到触发时刻”，已过则补发一次再重排。
 *
 * 主动消息格式（与普通回复一致，由 parseAndValidateReply 强制校验）：
 * 每条 <=1 句、连续表情 <=2，多条消息用换行分隔；话题临近结束时模型用短句/表情收尾。
 */

import {
  getChatRows,
  addChatRow,
  getSceneHistory,
  setScene,
  getConversationCompression,
  getSetting,
```

```
    setSetting
} from './storage.js'
import { memoryStore } from './chat-state.js'
import { getConversationSettings } from './chat-settings.js'
import { buildSystemPrompt, buildNowText, getPersonalityById } from './prompts.js'
import { chatCompletion } from './llm.js'
import { addLog } from './log.js'
import { emojiListForPrompt } from './emojis.js'
import { maybeCompress } from './chat-compress.js'
import { parseAndValidateReply } from './chat.js'
import { notifyProactive } from './notify.js'

// ----- 常量 -----

/** 频率档位 -> 随机间隔区间（分钟） */
const LEVEL_RANGES = {
  low: [45, 120],
  medium: [15, 45],
  high: [5, 15]
}

const TICK_MAX_MS = 60 * 60 * 1000 // 单次定时最长 60 分钟：即使间隔更长也先醒一次，保证窗口/设置变化及时响应
const HISTORY_ENTRIES = 15 // 主动消息注入的对话历史条数（上限，与 sendMessage 一致）
const PROACTIVE_DUE_KEY = 'proactive_due_at' // 到期时刻持久化键：进程结束（关页/杀进程）后重开时，据此感知“关闭期间已到触发时刻”并补发

// ----- 状态 -----

let _timer = null // 当前定时器
let _dueAt = 0 // 下一触发时刻（退后台期间保留，供回前台补发判断）
let _suppressed = false // 聊天页 loading 时抑制（等待 LLM 回复期间不打扰）
let _busy = false // 主动请求进行中（防止重入）

/** 读取持久化的到期时刻（绝对时间戳，无则 0） */
function _loadDueAt() {
  return parseInt(getSetting(PROACTIVE_DUE_KEY, 0), 10) || 0
}

/** 持久化到期时刻（进程结束后重开时用于判断是否错过触发；0 = 清空） */
function _saveDueAt(v) {
  try {
    setSetting(PROACTIVE_DUE_KEY, v)
  } catch (e) {
    /* ignore: 存储失败不影响调度本身 */
  }
}

function _clearTimer() {
  if (_timer) {
    clearTimeout(_timer)
  }
}
```

```
        _timer = null
    }
}

/** 聊天页同步 loading 状态: true 时调度器到期不发 (用户在等 LLM 回复) */
export function setProactiveSuppressed(v) {
    const was = _suppressed
    _suppressed = !!v
    // 解除抑制时若调度器已失活 (如 loading 期间到期未重排), 立即重排, 避免此后不再触发
    if (!_suppressed && was && !_timer) _schedule()
}

/**
 * 功能是否开启 (与瞬时抑制无关): 开关 + 现实时间 + API 配置齐备。
 * 调度重排只看此项----只要功能开着就持续重排, loading 抑制只拦截" 到期发送" 不拦截重排,
 * 保证任何时序下调度器都不会失活 (否则会表现为" 等待调度"、只能重启恢复)。
 */
function _featureOn(s) {
    return !! (
        s &&
        s.proactiveEnabled === true &&
        s.timeMode === 'real' &&
        s.apiKey &&
        s.baseUrl &&
        s.model
    )
}

/** 当前会话是否具备主动发送的基本条件 (功能开启 + 未被 loading 抑制) */
function _canRun(s) {
    return _featureOn(s) && !_suppressed
}

/** 最近一条用户消息时间 (无则 0) */
function _lastUserAt() {
    const rows = getChatRows()
    for (let i = rows.length - 1; i >= 0; i--) {
        if (rows[i].role === 'user') return Date.parse(rows[i].created_at) || 0
    }
    return 0
}

/** 当前时刻是否在主动消息时段窗口内 (分钟级, 闭区间 [start, end], start/end 为当天第几分钟 0-1439)
 */
function _inWindow(s) {
    const d = new Date()
    const min = d.getHours() * 60 + d.getMinutes()
    return min >= s.proactiveStartMin && min <= s.proactiveEndMin
}
```

```
/** 开关自定义倒计时 (调试值 >0 即处于" 测试模式"): 调试模式下忽略时段窗口与" 必须有用户消息" 门禁, 便于随时验证链路 */
```

```
function _isDebugCountdown(s) {  
    return parseInt(s && s.proactiveCustomSeconds, 10) > 0  
}
```

```
/** 主动消息被门禁拦截时写一条可见日志, 避免" 只见调度、不知为何不发" */
```

```
function _skipLog(reason) {  
    addLog('info', ' 主动消息跳过', reason)  
}
```

```
/** 当天第几分钟 -> "HH:MM" 展示 */
```

```
function _fmtClockMin(min) {  
    const h = String(Math.floor(min / 60)).padStart(2, '0')  
    const m = String(min % 60).padStart(2, '0')  
    return `${h}:${m}`  
}
```

```
/** 下次触发延迟 (ms): 调试用自定义倒计时 (proactiveCustomSeconds>0) 优先; 否则窗口外等到窗口起点、窗口内按档位随机间隔。始终 >=10s (自定义) 或 >=60s, 防忙循环/防误触 API 轰炸 */
```

```
function _nextDelay(s) {  
    const custom = parseInt(s && s.proactiveCustomSeconds, 10) || 0  
    if (custom > 0) return Math.max(10000, custom * 1000)  
    const d = new Date()  
    const min = d.getHours() * 60 + d.getMinutes()  
    const [lo, hi] = LEVEL_RANGES[s.proactiveLevel] || LEVEL_RANGES.medium  
    const toStart = (targetMin) => {  
        // 从当前时刻算起到 targetMin 这一分钟开头的毫秒数 (同分钟时为负, 代表" 已过该点")  
        return (targetMin - min) * 60000 - d.getSeconds() * 1000 - d.getMilliseconds()  
    }  
    if (min < s.proactiveStartMin) {  
        return Math.max(60000, toStart(s.proactiveStartMin))  
    }  
    if (min > s.proactiveEndMin) {  
        return Math.max(60000, toStart(s.proactiveStartMin + 1440))  
    }  
    return Math.max(60000, (lo + Math.random() * (hi - lo)) * 60000)  
}
```

```
// ----- 调度器 -----
```

```
/** 毫秒 -> "X 小时 Y 分" / "X 分 Y 秒" / "X 秒" (调度日志与倒计时展示用) */
```

```
function _fmtDuration(ms) {  
    const total = Math.max(0, Math.ceil((ms || 0) / 1000))  
    const h = Math.floor(total / 3600)  
    const m = Math.floor((total % 3600) / 60)  
    const s = total % 60  
    if (h > 0) return `${h} 小时 ${m} 分`  
    if (m > 0) return `${m} 分 ${String(s).padStart(2, '0')} 秒`  
    return `${s} 秒`  
}
```



```

}

/** 按当前会话设置安排下一次触发（清旧定时器后重排），并在调试日志记录调度状态。只看功能开关，抑制/
时段只拦发送不拦重排 */
function _schedule() {
  _clearTimer()
  const s = getConversationSettings()
  if (!_featureOn(s)) {
    // 功能关闭（关闭拟真/切虚拟时间/未配 API）：清掉持久化的到期时刻，避免残留值在日后重开时误
    触发

    _dueAt = 0
    _saveDueAt(0)
    return
  }
  const delay = _nextDelay(s)
  _dueAt = Date.now() + delay
  _saveDueAt(_dueAt)
  _timer = setTimeout(_tick, Math.min(delay, TICK_MAX_MS))
  addLog('info', ' 拟真聊天调度', `下次主动消息: ${_fmtDuration(delay)} 后 (${s.proactiveLevel} 档
)`)
}

/** 进程重启后恢复定时器：持久化到期时刻仍在未来-> 按剩余时间重挂；否则全新重排（或已补发过-> 重新起一
轮） */
function _armFromPersisted() {
  if (_dueAt && _dueAt > Date.now()) {
    _timer = setTimeout(_tick, Math.min(_dueAt - Date.now(), TICK_MAX_MS))
  } else {
    _schedule()
  }
}

function _tick() {
  _timer = null
  // _dueAt 为 0 说明已由 catchUp 补发过，此处不应再次触发（否则重复发送）
  if (_dueAt && Date.now() >= _dueAt) {
    _dueAt = 0
    // 到期触发：每次触发都必定产生一条可见日志----
    // 成功 -> 主动消息已发送（+ LLM 请求/响应）；
    // 被门禁拦截 -> 主动消息跳过（附原因），以及「拟真聊天到期．未发送」标记；
    // 异常 -> 主动消息失败（附堆栈），杜绝" 只见调度、无任何输出" 的静默失败。
    sendProactiveBurst()
    .then((r) => {
      if (r === null) addLog('info', ' 拟真聊天到期', ' 已到触发时刻但未发送（原因见「主动消
息跳过」日志）')
    })
    .catch((e) => addLog('err', ' 主动消息失败', (e && e.stack) || String(e)))
  }
  _schedule()
}

```

```
/**
 * 下次主动消息倒计时（毫秒）；未调度（功能未开启/尚未重排）返回 null。
 * 供设置页调试区实时显示调度状态。
 */
export function getProactiveCountdown() {
  if (!_timer || !_dueAt) return null
  const s = getConversationSettings()
  if (!_featureOn(s)) return null
  return Math.max(0, _dueAt - Date.now())
}

/**
 * 回前台/重开补发：若到期时刻已过（含进程结束后重开的场景），立即补发一次；随后恢复调度。
 * 幂等，可随时调用（切会话/改设置后用于让调度器立即按新状态重排）。
 */
export function catchUpProactive() {
  const s = getConversationSettings()
  if (!_featureOn(s)) {
    // 功能关闭（关闭拟真/切到虚拟时间/未配 API）：清掉在途定时器与持久化到期时刻
    _clearTimer()
    _dueAt = 0
    _saveDueAt(0)
    return
  }
  // 进程结束后内存 _dueAt 清零，但持久化的到期时刻仍在：据其判断" 关闭期间是否已到触发时刻"
  if (!_dueAt) _dueAt = _loadDueAt()
  if (_dueAt && Date.now() >= _dueAt) {
    // 清掉即将到期的旧定时器，避免其随后再触发一次造成重复发送，并清空持久化到期时刻
    _clearTimer()
    _dueAt = 0
    _saveDueAt(0)
    sendProactiveBurst()
    .then((r) => {
      if (r === null) addLog('info', ' 拟真聊天补发', ' 已到触发时刻但未发送（原因见「主动消息跳过」日志）')
    })
    .catch((e) => addLog('err', ' 主动消息失败', (e && e.stack) || String(e)))
  }
  if (!_timer) _armFromPersisted()
}

/**
 * 立即按当前会话设置重排下一次触发（修改倒计时/档位/时段/切会话后调用）。
 * 与 catchUpProactive 不同：即使定时器已挂载也强制重算（catchUp 为保持跨页倒计时不重置而跳过）。
 */
export function rearmProactive() {
  _schedule()
}
```

```
// ----- 主动消息发送 -----

/**
 * 发送一条主动消息 (拟真 burst): 组装 system (含拟真规则) + 最近历史, 调 LLM,
 * 校验后按换行拆多条落库 (Scene/Memory 由校验写入), 通知聊天页刷新。
 * @param {{force?: boolean}} [opts] force=true 用于调试按钮: 忽略时段/空闲/间隔门禁, 仅校验 API 配置
 * @returns {Promise<{lines:string[], saved:number}|null>} 未满足触发条件返回 null
 */
export async function sendProactiveBurst(opts = {}) {
  const force = !!opts.force
  const s = getConversationSettings()
  const debugCountdown = _isDebugCountdown(s)
  // 基本条件 (开关 + 现实时间 + 未抑制) 仅非调试时要求; 调试按钮忽略开关/时段/空闲/抑制, 但仍要求 API 配置
  if (!force && !_canRun(s)) {
    _skipLog(_suppressed ? '聊天页正在等待 LLM 回复 (已抑制), 本轮跳过' : '功能未开启 (需开启拟真 + 现实时间 + 已配置 API)')
    return null
  }
  if (!s.apiKey || !s.baseUrl || !s.model) {
    _skipLog('API 配置不完整 (缺少 baseUrl/API Key/model)')
    return null
  }
  // 触发门禁: 时段窗口 + 已有用户消息。自定义倒计时为调试工具, 忽略这两项门禁, 以便随时测试
  if (!force) {
    if (!debugCountdown && !_inWindow(s)) {
      _skipLog(`不在主动消息时段窗口 (${_fmtClockMin(s.proactiveStartMin)}-${_fmtClockMin(s.proactiveEndMin)})`)
      return null
    }
    if (!debugCountdown && !_lastUserAt()) {
      _skipLog('当前会话还没有用户消息, 本轮不主动发送')
      return null
    }
  }
  if (_busy) {
    _skipLog('上一条主动消息仍在处理中, 本轮跳过')
    return null
  }
  _busy = true
  try {
    // 组装 system (与 sendMessage 一致, proactive 注入拟真规则)
    const memoryText = memoryStore.retrieveContext('')
    const personalityPrompt =
      s.personalityId === 'custom' ? s.customPrompt : getPersonalityById(s.personalityId).prompt

    const system = buildSystemPrompt(
      personalityPrompt || '你是友好的聊天伙伴。',
      memoryText,

```

```

    getSceneHistory(),
    s.timeMode === 'virtual' ? '' : buildNowText(),
    s.emojiEnabled ? emojiListForPrompt() : [],
    memoryStore.l1Usage(),
    true
  )
}
const { summary, compressedUntil } = getConversationCompression()
const messages = [
  { role: 'system', content: summary ? `${system}\n\n 此前对话概要: ${summary}` : system }
]
for (const h of getChatRows().slice(compressedUntil).slice(-HISTORY_ENTRIES)) {
  messages.push({ role: h.role, content: h.content })
}
// 末尾追加内部指令 (仅请求, 不落库): 让模型以" 主动开口" 的方式输出下一条消息
messages.push({ role: 'user', content: ' (内部指令: 请作为 AI 主动开口给用户发消息, 只输出这
条主动消息本身) ' })

// 请求 -> 拟真格式校验 (每条 <=1 句、连续表情 <=2) -> 不合格自动重试
const maxAttempts = Math.max(1, Math.min(20, parseInt(s.maxRequestAttempts, 10) || 5))
let result = null
let lastReason = ''
for (let attempt = 1; attempt <= maxAttempts; attempt++) {
  const reply = await chatCompletion({
    baseUrl: s.baseUrl,
    apiKey: s.apiKey,
    model: s.model,
    messages,
    temperature: s.temperature,
    reasoningEffort: s.reasoningEffort
  })
  const parsed = parseAndValidateReply(reply.text, { proactive: true })
  if (parsed.ok) {
    result = parsed
    break
  }
  lastReason = parsed.reason
  addLog('info', ' 主动消息格式不合格, 自动重试', `第 ${attempt}/${maxAttempts} 次: ${lastReason}`)
}
if (!result) {
  addLog('err', ' 主动消息格式不合格 (已达请求上限)', `最大请求次数 ${maxAttempts}: ${lastReason}`)
  throw new Error(`主动消息格式连续 ${maxAttempts} 次不合格 (${lastReason})`)
}

// 按换行拆分多条落库 (每条 <=1 句一条气泡), rollback 挂最后一行
const lines = (result.cleanReply || '').split('\n').map((l) => l.trim()).filter(Boolean)
if (!lines.length) lines.push('...')
const sceneLenBefore = getSceneHistory().length
const rollback = { sceneLenBefore, memoryUndos: result.memoryUndos || [] }

```

```

    lines.forEach((line, idx) => {
      addChatRow('assistant', line, idx === lines.length - 1 ? { rollback } : undefined)
    })
    if (result.newScene) {
      setScene(result.newScene)
      addLog('info', ' 主动消息情景更新', result.newScene)
    }
    if (result.saved > 0) addLog('info', `主动消息记忆入库 ${result.saved} 条`)
    memoryStore.maintenance()
    maybeCompress().catch(() => { })
    addLog('info', ' 主动消息已发送', lines.join(' / '))
    // 系统通知：仅在应用处于后台时弹（前台刚发完用户正在看，不打扰）
    notifyProactive(lines.join('\n'))
    // 通知聊天页刷新（uni.$emit 在 App/H5/小程序均可用；Node 测试环境无此 API，跳过）
    if (typeof uni !== 'undefined' && uni.$emit) uni.$emit('proactive-burst', { lines })
    return { lines, saved: result.saved }
  } finally {
    _busy = false
  }
}

/** 调试按钮：立即发送一条主动消息（忽略时段/空闲/间隔门禁，仅校验 API 配置与开关） */
export async function debugProactiveMessage() {
  return sendProactiveBurst({ force: true })
}

// ===== FILE: utils/chat-settings.js =====
/**
 * 聊天服务 ---- 设置域：
 * 全局默认设置读取 / 会话设置快照读写（每个对话独享一份） / 会话人格保存
 */

import {
  getSetting,
  getConversationSettingsRaw,
  setConversationSettingsRaw,
  getConversationPersonality
} from './storage.js'
import { getPersonaName } from './prompts.js'
import { addLog } from './log.js'

/** 人格名称（含自定义，供 UI 展示） */
export function personaName(id) {
  return getPersonaName(id)
}

/** 全局默认设置（无设置快照的会话回退用；设置面板编辑的是当前会话设置） */
export function getSettings() {
  const personalityId = getSetting('personalityId', 'gentle')

```

```

return {
    baseUrl: getSetting('baseUrl', 'https://api.openai.com/v1'),
    apiKey: getSetting('apiKey', ''),
    model: getSetting('model', 'gpt-5.4-mini'),
    temperature: parseFloat(getSetting('temperature', '0.8')),
    reasoningEffort: getSetting('reasoningEffort', 'none'), // 思考模式: none 关闭 / high 开启 /
    ' 跟随模型 (Ollama 等兼容接口经 reasoning_effort 控制)'
    personalityId,
    customPrompt: getSetting('customPrompt', ''),
    timeMode: getSetting('timeMode', 'real'), // 情景时间模式: real 现实时间 / virtual 虚拟时间
    compressInterval: parseInt(getSetting('compressInterval', '0'), 10) || 0, // 自动压缩间隔 (
    条), 0= 关闭
    maxRequestAttempts: parseInt(getSetting('maxRequestAttempts', '5'), 10) || 5, // 回复格式不
    合格时的最大请求次数 (重试上限)
    emojiEnabled: getSetting('emojiEnabled', true) !== false, // 聊天表情包开关: 关闭后请求不携
    带表情清单
    // 语音阅读 (TTS): 默认关闭; 开启后 LLM 新回复会自动合成语音播放
    ttsEnabled: getSetting('ttsEnabled', false) !== false,
    ttsApiKey: getSetting('ttsApiKey', ''),
    ttsModel: getSetting('ttsModel', 'qwen3-tts-flash'),
    ttsVoice: getSetting('ttsVoice', 'Cherry'),
    // 拟真聊天: 仅现实时间模式生效; AI 在随机时间主动发消息 (每条 <=1 句、连续表情 <=2)
    proactiveEnabled: getSetting('proactiveEnabled', false),
    proactiveStartMin: getSetting('proactiveStartMin', 9 * 60), // 主动消息时段起 (当天第几分钟
    0-1439, 默认 09:00)
    proactiveEndMin: getSetting('proactiveEndMin', 23 * 60 + 59), // 主动消息时段止 (当天第几分
    钟 0-1439, 默认 23:59)
    proactiveLevel: getSetting('proactiveLevel', 'medium'), // 频率档位 low/medium/high
    proactiveCustomSeconds: parseInt(getSetting('proactiveCustomSeconds', '0'), 10) || 0, // 调
    试: 自定义倒计时 (秒), >0 时覆盖档位
    bubbleOpacity: (() => {
        const n = parseFloat(getSetting('bubbleOpacity', '1'))
        return Number.isFinite(n) ? Math.max(0.2, Math.min(1, n)) : 1
    })(), // 聊天气泡背景不透明度 0.2~1 (1= 不透明)
    personaName: getPersonaName(personalityId)
}
}

/** 归一化设置对象 (saveSettings / saveConversationPersonality / 新对话复制统一调用) */
export function normalizeSettings(s) {
    return {
        baseUrl: String(s && s.baseUrl ? s.baseUrl : '').trim(),
        apiKey: String(s && s.apiKey ? s.apiKey : '').trim(),
        model: String(s && s.model ? s.model : '').trim(),
        temperature: Number.isFinite(parseFloat(s && s.temperature)) ? parseFloat(s.temperature) : 0
        .8,
        reasoningEffort: (s && s.reasoningEffort) || '',
        personalityId: (s && s.personalityId) || 'gentle',
        customPrompt: String(s && s.customPrompt ? s.customPrompt : '').trim(),
        timeMode: s && s.timeMode === 'virtual' ? 'virtual' : 'real',

```

```

compressInterval: parseInt(s && s.compressInterval, 10) || 0,
maxRequestAttempts: Math.max(1, Math.min(20, parseInt(s && s.maxRequestAttempts, 10) || 5)),
emojiEnabled: !s || s.emojiEnabled !== false,
ttsEnabled: !s || s.ttsEnabled !== false,
ttsApiKey: String(s && s.ttsApiKey ? s.ttsApiKey : '').trim(),
ttsModel: String(s && s.ttsModel ? s.ttsModel : '').trim() || 'qwen3-tts-flash',
ttsVoice: String(s && s.ttsVoice ? s.ttsVoice : '').trim() || 'Cherry',
// 拟真聊天: 开关默认关; 时段为分钟级 (当天第几分钟 0-1439, 默认 09:00-23:59); 档位白名单
proactiveEnabled: !!s && s.proactiveEnabled === true,
proactiveStartMin: (() => {
  const n = parseInt(s && s.proactiveStartMin, 10)
  return Number.isNaN(n) ? 9 * 60 : Math.max(0, Math.min(1439, n))
})();
proactiveEndMin: (() => {
  const n = parseInt(s && s.proactiveEndMin, 10)
  return Number.isNaN(n) ? 23 * 60 + 59 : Math.max(0, Math.min(1439, n))
})();
proactiveLevel: ['low', 'medium', 'high'].includes(s && s.proactiveLevel) ? s.proactiveLevel
: 'medium',
// 自定义倒计时 (秒): 0= 用档位; 钳制在 0-3600, 供调试快速触发
proactiveCustomSeconds: (() => {
  const n = parseInt(s && s.proactiveCustomSeconds, 10)
  return Number.isNaN(n) ? 0 : Math.max(0, Math.min(3600, n))
})();
// 聊天气泡背景不透明度: 钳制 0.2~1, 1= 完全不透明
bubbleOpacity: (() => {
  const n = parseFloat(s && s.bubbleOpacity)
  return Number.isFinite(n) ? Math.max(0.2, Math.min(1, n)) : 1
})();
}
}

/** 保存设置到当前会话 (每个对话独享一份设置, 设置面板与之同步) */
export function saveSettings(s) {
  setConversationSettingsRaw(normalizeSettings(s))
}

/**
 * 当前会话生效设置 = 会话设置快照 (无快照回退全局; 旧数据兼容仅人格子集的快照)。
 * 每个会话独立一套完整设置, 切换会话即切换设置。
 */
export function getConversationSettings() {
  const s = getSettings()
  const raw = getConversationSettingsRaw()
  if (raw) {
    const merged = { ...s, ...raw }
    merged.temperature = Number.isFinite(parseFloat(raw.temperature)) ? parseFloat(raw.temperature) : s.temperature
    merged.compressInterval = parseInt(raw.compressInterval, 10) || 0
    merged.maxRequestAttempts = Math.max(1, Math.min(20, parseInt(raw.maxRequestAttempts, 10) ||

```

```

5))
    merged.ttsEnabled = raw.ttsEnabled === undefined ? s.ttsEnabled : raw.ttsEnabled !== false
    merged.personaName = getPersonaName(merged.personalityId || s.personalityId)
    return merged
}
// 旧数据：会话仅存人格子集 (personalityId/customPrompt/timeMode)
const p = getConversationPersonality()
return {
    ...s,
    personalityId: (p && p.personalityId) || s.personalityId,
    customPrompt: p && p.customPrompt !== undefined ? p.customPrompt : s.customPrompt,
    timeMode: (p && p.timeMode) || s.timeMode,
    personaName: getPersonaName((p && p.personalityId) || s.personalityId)
}
}

/**
 * 保存当前会话的人格设置（写入会话设置快照，仅作用于当前对话，不影响其他会话）。
 * timeMode 可选：不传则沿用当前会话生效的模式，保持会话内的情景时间设置不漂移。
 */
export function saveConversationPersonality(personalityId, customPrompt, timeMode) {
    const s = getConversationSettings()
    setConversationSettingsRaw(
        normalizeSettings({
            ...s,
            personalityId,
            customPrompt: personalityId === 'custom' ? (customPrompt || '') : '',
            timeMode: timeMode !== undefined ? timeMode : s.timeMode
        })
    )
    addLog('info', ' 会话人格', `${getPersonaName(personalityId)}${personalityId === 'custom' ? ' (自定义)' : ''}`)
}

// ===== FILE: utils/chat-state.js =====
/**
 * 聊天服务共享状态 ---- 供 chat.js / chat-conversations.js 等模块共享的可变状态：
 * - memoryStore: 记忆单例（会话切换重置 / 消息发送检索与入库 / 维护）
 * - lastRequest: 最近一次请求内容缓存（" 重新生成" 直接重发，不重复落库用户消息）
 * 独立成模块以消除 chat.js 与 chat-conversations.js 之间的循环依赖。
 */

import { MemoryStore } from './memory.js'

/** 记忆单例 */
export const memoryStore = new MemoryStore()

/** 最近一次请求内容缓存（跨模块读写） */
export const lastRequest = { value: '' }

```



```
// ===== FILE: utils/chat.js =====  
/**  
 * 聊天服务 ---- 门面 + 消息域：  
 * 对外统一导出（其余域拆分至 chat-settings / chat-conversations / chat-compress），  
 * 本文件承载发送主链路：检索记忆 -> 组装 system -> 调用 LLM -> 落库 -> 维护，  
 * 以及"重新生成"的撤回逻辑与回复格式校验。  
 */  
  
import {  
  initStorage,  
  getChatRows,  
  addChatRow,  
  truncateChat,  
  getSetting,  
  setSetting,  
  getSceneHistory,  
  setScene,  
  truncateSceneHistory,  
  getConversationCompression  
} from './storage.js'  
import { memoryStore, lastRequest } from './chat-state.js'  
import {  
  personaName,  
  getSettings,  
  saveSettings,  
  getConversationSettings,  
  saveConversationPersonality  
} from './chat-settings.js'  
import {  
  getHistoryForUI,  
  clearConversation,  
  listConversations,  
  activeConversationId,  
  startNewConversation,  
  openConversation,  
  removeConversation,  
  copyConversationToNew,  
  copyMemoriesToNew  
} from './chat-conversations.js'  
import { compressContext, maybeCompress } from './chat-compress.js'  
import { parseMemoryLine } from './memory.js'  
import { buildSystemPrompt, buildNowText, getPersonalityById } from './prompts.js'  
import { chatCompletion } from './llm.js'  
import { addLog } from './log.js'  
import { getEmojiMap, emojiListForPrompt, extractEmojiNames } from './emojis.js'  
  
// ----- 对外统一导出（门面） -----  
// chat.js 保持为唯一入口，各调用方（页面/工具/测试）的导入路径不变
```

```
export {
  memoryStore,
  personaName,
  getSettings,
  saveSettings,
  getConversationSettings,
  saveConversationPersonality,
  getHistoryForUI,
  clearConversation,
  listConversations,
  activeConversationId,
  startNewConversation,
  openConversation,
  removeConversation,
  copyConversationToNew,
  copyMemoriesToNew,
  compressContext,
  maybeCompress
}

// ----- 消息域 -----

const HISTORY_ENTRIES = 15 // 每轮注入的对话历史条数 (上限)
const MAINTENANCE_INTERVAL_MS = 30 * 60 * 1000 // 维护节流间隔 (30 分钟)

/** 初始化存储 (幂等, App.vue onLaunch 调用) */
export function initChatService() {
  initStorage()
  memoryStore.maintenance()
  setSetting('lastMaintenance', new Date().toISOString())
}

/**
 * 节流维护: 距离上次维护超过间隔才执行 (App 进入前台 / 页面 onShow 调用)。
 * 对话后的主动维护仍走 memoryStore.maintenance(), 不受此限制。
 */
export function maybeMaintenance() {
  const last = getSetting('lastMaintenance', '')
  if (last) {
    const t = Date.parse(last)
    if (!Number.isNaN(t) && Date.now() - t < MAINTENANCE_INTERVAL_MS) return
  }
  memoryStore.maintenance()
  setSetting('lastMaintenance', new Date().toISOString())
}

/**
 * 用户点" 重新生成" 时调用: 移除待重发请求对应的最后一条助手回复 (撤回其情景与记忆)。
 * 返回待重发内容----优先取最近一次请求缓存, 无缓存 (如重启后) 回退到当前会话最后一条用户消息;
```

```

* 若无可重发内容则返回空串。用户消息保留在存储中，重发 (persistUser:false) 不再重复落库。
*/
export function popLastAssistant() {
  const rows = getChatRows()
  // 待重发内容：最近一次请求缓存优先；无缓存（如重启后）回退到最后一条用户消息
  let req = lastRequest.value
  if (!req) {
    for (let i = rows.length - 1; i >= 0; i--) {
      if (rows[i].role === 'user') {
        req = rows[i].content
        break
      }
    }
  }
  if (!req) return ''
  // 定位该请求对应的最后一条用户消息（从后往前匹配，取最近一次），
  // 若其后有 assistant 回复则移除并撤回其情景/记忆（拟真模式 burst 为连续多条 assistant 行，
  // 一并移除，rollback 挂在最后一行）；发送失败时无回复可移，仅重发
  for (let i = rows.length - 1; i >= 0; i--) {
    if (rows[i].role === 'user' && rows[i].content === req) {
      let last = i
      while (last + 1 < rows.length && rows[last + 1].role === 'assistant') last++
      if (last > i) {
        rollbackAssistantEffects(rows[last])
        truncateChat(i + 1)
      }
      break
    }
  }
  addLog('info', ' 重新生成', `重发最近一次请求: ${req.slice(0, 20)}`)
  return req
}

/** 撤回一条响应记录附带的情景与记忆变更（重新生成前调用） */
function rollbackAssistantEffects(row) {
  const rb = row && row.rollback
  if (!rb) return
  // 情景：截断到该响应之前的情景历史长度，撤掉本响应新增的情景
  if (typeof rb.sceneLenBefore === 'number') {
    truncateSceneHistory(rb.sceneLenBefore)
  }
  // 记忆：按写入顺序逆序撤销（新增-> 删除 / 修改-> 恢复原值 / 删除-> 重新插入）
  const undos = rb.memoryUndos
  if (Array.isArray(undos) && undos.length) {
    for (let i = undos.length - 1; i >= 0; i--) memoryStore.applyUndo(undos[i])
  }
  addLog('info', ' 撤回响应情景与记忆', `情景截断到 ${rb.sceneLenBefore}, 撤销记忆 ${undos && undos.length} || 0} 条`)
}

```

```
/**
 * 发送一条用户消息
 * @param {string} userText
 * @param {{persistUser?: boolean}} [opts] persistUser=false 表示"重新生成"重发:
 *      用户消息已在首次发送时落库, 本次只更新回复, 避免重复记录同一句话
 * @returns {Promise<{reply:string, saved:number}>} reply 为清理掉 Memory 行后的回复
 */
export async function sendMessage(userText, opts = {}) {
  // 设置全部取当前会话 (每个对话独享一份设置: API 配置 / 人格 / 时间模式等)
  const persistUser = opts.persistUser !== false
  const s = getConversationSettings()
  if (!s.apiKey) throw new Error(' 请先在「设置」中填写 API Key')
  if (!s.baseUrl) throw new Error(' 请先在「设置」中填写接口地址')
  if (!s.model) throw new Error(' 请先在「设置」中填写模型名称')
  // 拟真聊天: 该会话开启后, 所有回复均受" 每条 <=1 句、连续表情 <=2" 约束, 并按换行拆分为多条气泡
  const proactive = !!s.proactiveEnabled

  // 缓存最近一次请求: 聊天页" 重新生成" 直接重发该内容 (重发时 persistUser:false, 不重复落库用户消息)
  lastRequest.value = userText

  // 记录发送前的情景历史长度: 若本响应更新了情景, 重新生成时据此撤回
  const sceneLenBefore = getSceneHistory().length

  // 1. 检索记忆 (全量 L1 + 新鲜槽 + MMR 多样性槽)
  const memoryText = memoryStore.retrieveContext(userText)
  addLog('info', ' 发送消息', userText.slice(0, 40))

  // 2. 组装 system prompt (人格 + 规则 + 记忆指南 + 情景指南 + 当前状态 + 记忆 + L1 容量 + 表情包清单)
  const personalityPrompt =
    s.personalityId === 'custom' ? s.customPrompt : getPersonalityById(s.personalityId).prompt
  const system = buildSystemPrompt(
    personalityPrompt || ' 你是友好的聊天伙伴。',
    memoryText,
    getSceneHistory(),
    s.timeMode === 'virtual' ? '' : buildNowText(),
    // 表情包开关: 禁用时不传清单 (buildSystemPrompt 不注入表情包规则, LLM 不会主动使用表情)
    s.emojiEnabled ? emojiListForPrompt() : [],
    // L1 容量状态: 引导 LLM 在 L1 满员时先逐字删除/修改旧 L1 再新增
    memoryStore.l1Usage(),
    // 拟真聊天: 注入每条消息 <=1 句的规则 (开启时生效)
    proactive
  )

  // 3-5. 请求 -> 格式校验 -> 解析: 回复格式不合格时自动重试 (上限 maxRequestAttempts)。
  // 每次尝试都基于同一份历史重新组装; 校验通过才写入 Scene/Memory 并落库, 失败尝试无副作用。
  const maxAttempts = Math.max(1, Math.min(20, parseInt(s.maxRequestAttempts, 10) || 5))
  let result = null
  let lastReason = ''
```

```

// 任一尝试失败（网络错误 / 格式校验达上限）都只记录用户请求、不落库错误回复，
// 聊天页" 重新生成" 基于最近一次请求缓存直接重发
try {
  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    // 组装 messages: system + 未压缩历史 + 当前用户消息
    // 压缩概要并入首条 system 末尾，避免出现第二条 system 消息----Ollama 等模板要求
    // system 必须在 messages 最前且只能一条，多条 system 会抛 "System message must be at the beginning"
    const { summary, compressedUntil } = getConversationCompression()
    const messages = [
      { role: 'system', content: summary ? `${system}\n\n 此前对话概要: ${summary}` : system }
    ]
    for (const h of getChatRows().slice(compressedUntil).slice(-HISTORY_ENTRIES)) {
      messages.push({ role: h.role, content: h.content })
    }
    messages.push({ role: 'user', content: userText })

    const reply = await chatCompletion({
      baseUrl: s.baseUrl,
      apiKey: s.apiKey,
      model: s.model,
      messages,
      temperature: s.temperature,
      reasoningEffort: s.reasoningEffort // 默认关闭思考 (none)，本地思考模型（如 Qwen3.5
    })
    // 默认思考会占满输出 token 导致回复为空
    })

    const parsed = parseAndValidateReply(reply.text, { proactive })
    if (parsed.ok) {
      result = parsed
      break
    }
    lastReason = parsed.reason
    addLog('info', ' 回复格式不合格，自动重试', `第 ${attempt}/${maxAttempts} 次: ${lastReason}`)
  }
  if (!result) {
    addLog('err', ' 回复格式不合格（已达请求上限）', `最大请求次数 ${maxAttempts}: ${lastReason}`)
    throw new Error(`模型回复格式连续 ${maxAttempts} 次不合格 (${lastReason})`)
  }
} catch (e) {
  // 发送失败：只落库用户请求（不落库错误回复），供" 重新生成" 定位本次请求
  if (persistUser) addChatRow('user', userText)
  throw e
}

// 6. 落库对话：落库清理后的文本，Scene/Memory 标记不再混入历史----否则从其它页切回时
// 重新加载历史会把标记显示在气泡里（此前 bug），也会污染上下文压缩。

```

```

// assistant 行附带 rollback 信息 (响应前的情景长度 + 本响应的记忆撤销清单),
// 重新生成时据此撤回该响应记录的情景与记忆。
// 拟真模式: 按换行拆分多条落库 (每条 <=1 句一条气泡), rollback 挂在最后一行
if (persistUser) addChatRow('user', userText)
const cleanReply = result.cleanReply || '...'
const rollback = { sceneLenBefore, memoryUndos: result.memoryUndos || [] }
if (proactive) {
  const lines = cleanReply.split('\n').map((l) => l.trim()).filter(Boolean)
  if (!lines.length) lines.push('...')
  lines.forEach((line, idx) => {
    addChatRow('assistant', line, idx === lines.length - 1 ? { rollback } : undefined)
  })
} else {
  addChatRow('assistant', cleanReply, { rollback })
}
if (result.newScene) {
  setScene(result.newScene)
  addLog('info', ' 情景更新', result.newScene)
}
if (result.saved > 0) addLog('info', `记忆入库 ${result.saved} 条`)

// 7. 定期维护 (L3 过期清理 / 降级 / 容量控制)
memoryStore.maintenance()

// 8. 自动压缩检查 (异步执行, 不阻塞回复展示)
maybeCompress().catch(() => { })

return { reply: cleanReply, saved: result.saved, burst: proactive ? cleanReply.split('\n').map((
1) => l.trim()).filter(Boolean) : undefined }
}

/**
 * 统计文本句子数: 按句末标点 (。!?!?... ) 计, 连续标点 (如" 哈哈。。") 合并为一处,
 * 用于拟真聊天" 每条消息 <=1 句" 的校验。
 * @param {string} text
 * @returns {number}
 */
export function countSentences(text) {
  const t = String(text || '').trim()
  if (!t) return 0
  let runs = 0
  let inRun = false
  for (const ch of t) {
    if ('。!?!?...'.includes(ch)) {
      if (!inRun) {
        runs++
        inRun = true
      }
    } else {
      inRun = false
    }
  }
}

```

```

    }
  }
  return runs
}

/**
 * 校验 LLM 回复格式并解析 Scene/Memory (校验通过才写库, 供 sendMessage 重试使用):
 * - 回复必须包含对话文本 (仅 Scene/Memory 标记视为不合格)
 * - Scene 行必须有内容; Memory 行必须能被 parseMemoryLine 解析
 * - 拟真模式 (proactive:true): 每条文本行 <=1 句、连续表情 <=2, 不合格触发重试
 * @param {string} text LLM 原始回复
 * @param {{proactive?: boolean}} [opts]
 * @returns {{ok:boolean, reason?:string, saved?:number, newScene?:string|null, cleanReply?:string,
memoryUndos?:Array}}
 */
export function parseAndValidateReply(text, opts) {
  const proactive = !!opts && opts.proactive
  // 表情名校验: 回复中的 $ 名 $ 必须存在于表情清单, 否则判定格式不合格触发重试
  // (避免 LLM 编造清单外的名字导致聊天网页渲染出"坏图")
  const emojiMap = getEmojiMap()
  for (const name of extractEmojiNames(text)) {
    if (!emojiMap[name]) return { ok: false, reason: `回复包含未知表情名: ${name}` }
  }
  const lines = String(text || '').split('\n')
  // 第一步: 格式校验 (不写入)
  let newScene = null
  let hasContent = false
  const cleanLines = []
  for (const line of lines) {
    if (/^s*Scene\s*[: ]/i.test(line)) {
      newScene = line
        .replace(/^s*Scene\s*[: ]s*/i, '')
        .replace(/^["'"]+|["'"]+$/g, '') // 去掉 LLM 可能加上的引号
        .replace(/[\.\!\ ]$/g, '') // 去掉末尾标点
        .trim()
      if (!newScene) return { ok: false, reason: 'Scene 行内容为空' }
      continue
    }
    if (/^s*Memory\s*[: ]/i.test(line)) {
      if (!parseMemoryLine(line)) return { ok: false, reason: `Memory 行格式非法: ${line.trim().slice(0, 24)}` }
      continue
    }
    // 行内 Scene/Memory 标记 (如 "正文 [Scene: xxx]"): Scene/Memory 必须独立成行,
    // 行首形式已在上方分支处理, 走到这里仍含 Scene:/Memory: 即未独立成行 -> 判定格式非法
    if (/Scene\s*[: ]/i.test(line) || /Memory\s*[: ]/i.test(line)) {
      return { ok: false, reason: `Scene/Memory 标记未独立成行: ${line.trim().slice(0, 28)}` }
    }
    // 疑似 Memory 行: 含 | keywords: | importance: | level: 分段结构却缺少 Memory: 前缀
    // (如 "[OK] 更新: 3. xxx | keywords:... | importance:... | level:..."), 判定格式非法并触发重

```

试

```

    if (/\\s*(?:keywords|importance|level):/i.test(line)) {
        return { ok: false, reason: `疑似 Memory 行缺少 Memory: 前缀: ${line.trim().slice(0, 28)}
    }` }

    }

    // 拟真模式: 每条消息 (一行) <=1 句、连续表情 <=2, 超出即判定格式不合格触发重试
    if (proactive) {
        if (countSentences(line) > 1) {
            return { ok: false, reason: `拟真模式消息超过一句话: ${line.trim().slice(0, 20)}` }
        }
        if (extractEmojiNames(line).length > 2) {
            return { ok: false, reason: `拟真模式连续表情超过 2 个: ${line.trim().slice(0, 20)}`
        }

        }

        }

        cleanLines.push(line)
        if (line.trim()) hasContent = true
    }

    if (!hasContent) return { ok: false, reason: ' 回复仅含 Scene/Memory 标记, 无对话文本' }
    const cleanReply = cleanLines.join('\n').trim()

    // 第二步: 校验通过, 执行写入 (Scene 更新情景 / Memory 入库), 并收集撤销信息供重新生成回滚
    let saved = 0
    const memoryUndos = []
    for (const line of lines) {
        if (/^\\s*Memory\\s*[: ]/i.test(line)) {
            const undo = memoryStore.saveFromLine(line)
            if (undo) {
                saved++
                memoryUndos.push(undo)
            }
        }
    }

    return { ok: true, saved, newScene, cleanReply, memoryUndos }
}

// ===== FILE: utils/emojis.js =====
/**
 * 表情包数据层 ---- 全局共享的表情列表 (不随会话切换)
 *
 * 每个表情 { id, name, src, created_at }:
 * - name 表情名: 上传时必填、全局唯一、<=20 字、不含 $/换行; 消息中用 $ 表情名 $ 引用
 * - src 图片: App / 微信小程序为 uni.saveFile 持久化路径, H5 为压缩后的 PNG base64
 *
 * 存储键 chabot_emojis (uni storage, 跨端可用)。表情数量与图片大小设上限,
 * 避免 H5 localStorage 配额 (约 5MB) 被大图打满。
 */

const KEY_EMOJIS = 'chabot_emojis'
```



```
export const EMOJI_MAX_COUNT = 50 // 表情数量上限
export const EMOJI_MAX_NAME_LEN = 20 // 表情名长度上限 (字), 与 $ 名 $ 占位解析的正则上限一致
const EMOJI_IMG_MAX_WIDTH = 300 // H5 压缩后的图片最大宽度 (px)
const EMOJI_IMG_MAX_BYTES = 300 * 1024 // H5 base64 单图大小上限 (约 300KB)

let _emojis = [] // 内存缓存 (活数据)
let _loaded = false

/** 首次访问时从存储加载 (幂等) */
function _ensureLoaded() {
  if (_loaded) return
  _loaded = true
  if (typeof uni !== 'undefined' && uni.getStorageSync) {
    const v = uni.getStorageSync(KEY_EMOJIS)
    _emojis = Array.isArray(v) ? v.filter((e) => e && typeof e.name === 'string' && e.name && e.src) : []
  } else {
    _emojis = []
  }
}

function _persist() {
  if (typeof uni !== 'undefined' && uni.setStorageSync) {
    try {
      uni.setStorageSync(KEY_EMOJIS, _emojis)
    } catch (e) {
      console.error('[emojis] 保存失败:', e)
    }
  }
}

/** 表情列表 (副本) */
export function getEmojis() {
  _ensureLoaded()
  return _emojis.map((e) => ({ ...e }))
}

/** 表情名 -> 图片 映射 (渲染与校验用) */
export function getEmojiMap() {
  _ensureLoaded()
  const map = {}
  for (const e of _emojis) map[e.name] = e.src
  return map
}

/** 表情名清单 (供 LLM 提示词注入) */
export function emojiListForPrompt() {
  _ensureLoaded()
  return _emojis.map((e) => e.name)
}
```

```
}

/**
 * 校验表情名：必填 / <=20 字 / 不含 $ 与换行 / 全局唯一（改名时排除自身）
 * @param {string} name
 * @param {string} [excludeId] 排除的表情 id（改名时传入自身）
 * @returns {{ok:boolean, reason?:string}}
 */
export function validateEmojiName(name, excludeId) {
  const n = String(name || '').trim()
  if (!n) return { ok: false, reason: '请填写表情名' }
  if (n.length > EMOJI_MAX_NAME_LEN) return { ok: false, reason: `表情名不能超过 ${EMOJI_MAX_NAME_LEN} 字` }
  if (n.indexOf('$') !== -1 || /\r\n/.test(n)) return { ok: false, reason: '表情名不能包含 $ 或换行' }
  _ensureLoaded()
  for (const e of _emojis) {
    if (e.name === n && e.id !== excludeId) return { ok: false, reason: `表情名「${n}」已存在` }
  }
  return { ok: true }
}

/**
 * 以已持久化的 src 直接新增表情（图片保存由 addEmoji 负责；测试/导入场景可直用）
 * @param {string} name 表情名
 * @param {string} src 图片路径 / base64
 * @returns {{id:string, name:string, src:string, created_at:string}} 新增的表情副本
 */
export function addEmojiData(name, src) {
  const v = validateEmojiName(name)
  if (!v.ok) throw new Error(v.reason)
  _ensureLoaded()
  if (_emojis.length >= EMOJI_MAX_COUNT) throw new Error(`表情数量已达上限 (${EMOJI_MAX_COUNT} 个)`)
  if (!src) throw new Error('缺少表情图片')
  const item = {
    id: 'e' + Date.now().toString(36) + '_' + Math.floor(Math.random() * 1e4),
    name: name.trim(),
    src,
    created_at: new Date().toISOString()
  }
  _emojis.push(item)
  _persist()
  return { ...item }
}

/** 保存表情图片为持久化 src（App/小程序压缩后存文件，H5 转小尺寸 PNG base64），失败返回空串 */
async function _saveEmojiImage(tempPath) {
  let saved = ''
  // #ifdef APP-PLUS || MP-WEIXIN
```

// 上传时先压缩 (限宽 + quality), 避免原图过大: 消息列表渲染大量大图会明显拖慢 App 启动与滚动
// 注意: compressImage 输出为 jpg, PNG 透明底会变白, 表情多为此可接受; H5 端仍走 PNG base64 保留透明

```
if (typeof uni.compressImage === 'function') {
  const compressed = await new Promise((resolve) => {
    uni.compressImage({
      src: tempPath,
      quality: 80,
      compressedWidth: EMOJI_IMG_MAX_WIDTH,
      success: (res) => resolve(res.tempFilePath || ''),
      fail: () => resolve('')
    })
  })
  if (compressed) tempPath = compressed
}
saved = await new Promise((resolve) => {
  uni.saveFile({
    tempFilePath: tempPath,
    success: (res) => resolve(res.savedFilePath || ''),
    fail: () => resolve('')
  })
})
// #endif
// #ifdef H5
saved = await _h5EmojiToBase64(tempPath)
// #endif
return saved
}

// #ifdef H5
/** H5 端: canvas 把图片压缩为小尺寸 PNG base64 (保留透明, 限制宽度与体积) */
function _h5EmojiToBase64(tempPath) {
  return new Promise((resolve) => {
    const img = new Image()
    img.onload = () => {
      try {
        let w = img.naturalWidth || img.width
        let h = img.naturalHeight || img.height
        if (w > EMOJI_IMG_MAX_WIDTH) {
          h = Math.round((h * EMOJI_IMG_MAX_WIDTH) / w)
          w = EMOJI_IMG_MAX_WIDTH
        }
        const canvas = document.createElement('canvas')
        canvas.width = w
        canvas.height = h
        canvas.getContext('2d').drawImage(img, 0, 0, w, h)
        const dataUrl = canvas.toDataURL('image/png')
        // base64 长度 x 3/4 ~ 字节数, 超限视为失败 (提示重新选择更小图片)
        if (dataUrl.length * 0.75 > EMOJI_IMG_MAX_BYTES) return resolve('')
        resolve(dataUrl)
      }
    }
  })
}
```

```
        } catch (e) {
            console.error('[emojis] 图片压缩失败', e)
            resolve('')
        }
    }
    img.onerror = () => resolve('')
    img.src = tempPath
})
}
// #endif

/**
 * 选择图片后新增表情 (校验 -> 保存图片 -> 落库)
 * @param {string} tempPath uni.chooseImage 返回的临时路径
 * @param {string} name 表情名
 * @returns {Promise<object>} 新增的表情副本
 */
export async function addEmoji(tempPath, name) {
    const v = validateEmojiName(name)
    if (!v.ok) throw new Error(v.reason)
    if (!tempPath) throw new Error(' 未选择图片')
    const src = await _saveEmojiImage(tempPath)
    if (!src) throw new Error(' 图片保存失败, 请换一张图片重试')
    return addEmojiData(name, src)
}

/** 修改表情名 (唯一性校验排除自身) */
export function renameEmoji(id, newName) {
    const v = validateEmojiName(newName, id)
    if (!v.ok) throw new Error(v.reason)
    _ensureLoaded()
    const e = _emojis.find((x) => x.id === id)
    if (!e) throw new Error(' 表情不存在')
    e.name = newName.trim()
    _persist()
    return { ...e }
}

/** 删除表情 (顺带清理已保存的图片文件), 不存在返回 false */
export function deleteEmoji(id) {
    _ensureLoaded()
    const idx = _emojis.findIndex((e) => e.id === id)
    if (idx < 0) return false
    const [removed] = _emojis.splice(idx, 1)
    _persist()
    if (removed.src) {
        // #ifdef APP-PLUS || MP-WEIXIN
        uni.removeSavedFile({ filePath: removed.src, fail: () => { } })
        // #endif
    }
}
```

```

    return true
}

/**
 * 按新顺序重排表情列表（拖拽排序用），列表外的 id 自动忽略。
 * @param {string[]} orderedIds 新顺序的表情 id 数组
 */
export function reorderEmojis(orderedIds) {
    _ensureLoaded()
    const byId = new Map(_emojis.map((e) => [e.id, e]))
    const next = (Array.isArray(orderedIds) ? orderedIds : [])
        .map((id) => byId.get(id))
        .filter(Boolean)
    // 补上未出现在新顺序里的表情（防御：保证列表不丢项）
    for (const e of _emojis) {
        if (!next.includes(e)) next.push(e)
    }
    _emojis = next
    _persist()
}

/** 新建正则：匹配 $ 表情名 $ 占位（名 1~20 字，不含 $ 与换行） */
function _emojiTokenRe() {
    return /\$([^\n]{1,20})\$/g
}

/** 提取文本中所有 $ 表情名 $ 占位名（含清单外的未知名），供 LLM 回复校验使用 */
export function extractEmojiNames(text) {
    const names = []
    const re = _emojiTokenRe()
    let m
    while ((m = re.exec(String(text || ''))) !== null) names.push(m[1])
    return names
}

/**
 * 把消息文本按 $ 表情名 $ 拆分为段数组（纯函数，供消息渲染与测试）。
 * 表情包前后的空格/换行等纯空白段会被跳过，文本段去除首尾空白----
 * 避免模型在表情之间插入空格/换行时渲染出空消息气泡。
 * @param {string} content 原始消息（可含 $ 名 $ 占位）
 * @param {Object} [map] 表情名 -> 图片（getEmojiMap() 的结果）
 * @returns {Array<{type:'text', text:string}|{type:'emoji', name:string, src:string}>}
 *      清单外的未知 $ 名 $ 原样保留在文本段中
 */
export function splitEmojiText(content, map) {
    const text = String(content || '')
    const out = []
    const re = _emojiTokenRe()
    let last = 0
    let m

```

```
while ((m = re.exec(text)) !== null) {
  const name = m[1]
  const seg = text.slice(last, m.index)
  // 表情前的文本段：纯空白跳过，非空白去除首尾空白
  const trimmed = seg.trim()
  if (trimmed) out.push({ type: 'text', text: trimmed })
  const src = map && map[name]
  out.push(src ? { type: 'emoji', name, src } : { type: 'text', text: m[0] })
  last = re.lastIndex
}
if (last < text.length) {
  const trimmed = text.slice(last).trim()
  if (trimmed) out.push({ type: 'text', text: trimmed })
}
return out
}

// ===== FILE: utils/export.js =====
/**
 * 聊天记录导出 ---- 把当前会话聊天记录组装为纯文本，按平台导出一键 .txt:
 * - H5: Blob 直接触发浏览器下载（带 BOM 防中文乱码）
 * - App: plus.io 写入应用文档目录（_doc），并尝试系统打开
 * - 微信小程序：小程序无法直接导出 txt 文件，降级为复制全文到剪贴板
 */

import { getChatRows, getConversations, getActiveConversationId } from './storage.js'
import { getConversationSettings } from './chat.js'

function _pad(n) {
  return String(n).padStart(2, '0')
}

function _nowText() {
  const d = new Date()
  return `${d.getFullYear()}-${_pad(d.getMonth() + 1)}-${_pad(d.getDate())} ${_pad(d.getHours())}:
${_pad(d.getMinutes())}`
}

/** 组装当前会话聊天记录文本（纯函数，供导出与测试） */
export function buildChatExportText() {
  const rows = getChatRows()
  if (!rows.length) return ''
  const conv = getConversations().find((c) => c.id === getActiveConversationId())
  const s = getConversationSettings()
  const lines = []
  lines.push(`${s.personaName} . ${conv && conv.title} || ' 新对话'`)
  lines.push(`导出时间: ${_nowText()}`)
  lines.push('-----')
  for (const r of rows) {
```

```
.free-tip {
  font-size: 22rpx;
  color: var(--c-text-aid);
  line-height: 1.6;
  margin-bottom: 16rpx;
}

.free-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 20rpx 24rpx;
  border-radius: 12rpx;
  background: var(--c-bg);
  margin-bottom: 16rpx;
  border: 2rpx solid transparent;
}

.free-item:last-child {
  margin-bottom: 0;
}

.free-item.on {
  border-color: var(--c-primary);
  background: var(--c-primary-light);
}

.free-main {
  flex: 1;
  min-width: 0;
  margin-right: 16rpx;
}

.free-name {
  font-size: 28rpx;
  color: var(--c-text);
  font-weight: 500;
}

.free-desc {
  font-size: 22rpx;
  color: var(--c-text-aid);
  margin-top: 6rpx;
  line-height: 1.5;
}

.free-btn {
  flex-shrink: 0;
  font-size: 24rpx;
  color: var(--c-primary);
  border: 1rpx solid var(--c-primary);
  border-radius: 26rpx;
  padding: 8rpx 26rpx;
}

.free-item.on .free-btn {
  color: #fff;
}
```

```
        background: var(--c-primary);
    }

    /* 配置教学弹窗 */
    .mask {
        position: fixed;
        inset: 0;
        background: rgba(0, 0, 0, 0.45);
        z-index: 999;
        display: flex;
        align-items: center;
        justify-content: center;
    }
    .tutorial {
        width: 640rpx;
        max-height: 80vh;
        background: var(--c-card);
        border-radius: 24rpx;
        padding: 32rpx;
        display: flex;
        flex-direction: column;
    }
    .tutorial-title {
        font-size: 32rpx;
        font-weight: 600;
        color: var(--c-text);
        margin-bottom: 20rpx;
    }
    .tutorial-scroll {
        flex: 1;
        min-height: 0;
        max-height: 60vh;
    }
    .t-step {
        margin-bottom: 24rpx;
    }
    .t-step-title {
        font-size: 26rpx;
        color: var(--c-primary);
        font-weight: 600;
        margin-bottom: 6rpx;
    }
    .t-step-body {
        font-size: 24rpx;
        color: var(--c-text-secondary);
        line-height: 1.7;
    }
    .tutorial-close {
        margin-top: 20rpx;
        height: 80rpx;
    }
```



```
    line-height: 80rpx;
    text-align: center;
    font-size: 28rpx;
    color: #fff;
    background: var(--c-brand-gradient);
    border-radius: 999rpx;
  }
</style>

// ===== FILE: pages/settings/appearance.vue =====
<template>
  <view class="ss-page" :class="themeClass">
    <view class="ss-tipbar"> 界面外观：包含深色模式、聊天气泡不透明度与聊天背景。更改后点击底部「
    保存设置」生效（深色模式与背景即时生效）。</view>

    <view class="ss-card">
      <view class="ss-title"> 深色模式 </view>
      <view class="ss-row">
        <view class="ss-row-head">
          <text class="ss-label" style="margin-bottom: 0"> 切换深色 / 浅色 </text>
          <view class="ss-btns">
            <view class="ss-btn first" :class="{ on: __theme === 'light' }" @tap="pickTh
            eme('light')"> 浅色 </view>
            <view class="ss-btn" :class="{ on: __theme === 'dark' }" @tap="pickTheme('da
            rk')"> 深色 </view>
          </view>
        </view>
        <view class="ss-hint">
          {{ __theme === 'dark' ? ' 当前为深色模式' : ' 当前为浅色模式' }}, 即时生效
        </view>
      </view>
    </view>

    <view class="ss-card">
      <view class="ss-title"> 聊天气泡 </view>
      <view class="ss-row">
        <text class="ss-label"> 背景不透明度 {{ Math.round((s.bubbleOpacity || 1) * 100) }}%<
        /text>
        <slider :value="s.bubbleOpacity" :min="0.2" :max="1" :step="0.05" activeColor="var(--
        c-primary)" @change="onBubbleOpacity" />
        <view class="ss-hint"> 数值越低气泡背景越透明、越能看到下方背景图；文字始终清晰清晰。
        此设置随对话保存。</view>
      </view>
    </view>

    <view class="ss-card">
      <view class="ss-title"> 聊天背景 </view>
      <view v-if="bg" class="bg-preview" :style="bgStyle">
        <text class="bg-preview-tip"> 当前背景 </text>
      </view>
    </view>
  </view>
</template>
```

```
<view class="bg-actions">
  <button class="bg-btn primary" @tap="chooseBg"> 选择图片 </button>
  <button v-if="bg" class="bg-btn" @tap="removeBg"> 移除背景 </button>
</view>
<view class="ss-hint"> 选择后即时应用到聊天页并覆盖背景，立即生效、无需保存。</view>
</view>

<view class="ss-save" @tap="save"> 保存设置 </view>
</view>
</template>

<script>
import { getConversationSettings, saveSettings } from '../utils/chat.js'
import { getBackgroundImage, saveBackgroundImage, removeBackgroundImage } from '../utils/storage.js'
import { setTheme } from '../utils/theme.js'
import { addLog } from '../utils/log.js'

export default {
  data() {
    return {
      s: getConversationSettings(),
      bg: getBackgroundImage()
    }
  },
  computed: {
    bgStyle() {
      return this.bg
        ? {
            backgroundImage: "url('" + this.bg + "')",
            backgroundSize: 'cover',
            backgroundPosition: 'center',
            backgroundRepeat: 'no-repeat'
          }
        : {}
    }
  },
  onShow() {
    this.s = getConversationSettings()
    this.bg = getBackgroundImage()
  },
  methods: {
    pickTheme(t) {
      setTheme(t) // 全局即时生效
    },
    onBubbleOpacity(e) {
      this.s.bubbleOpacity = e.detail.value
    },
    save() {
      saveSettings(this.s) // 保存气泡不透明度（随会话快照）
    }
  }
}
```

```
        addLog('info', ' 保存设置', ' 界面外观 (气泡不透明度) ')
        uni.showToast({ title: ' 已保存', icon: 'success' })
    },
    chooseBg() {
        uni.chooseImage({
            count: 1,
            sizeType: ['compressed'],
            success: async (res) => {
                const tempPath = res.tempFilePaths[0]
                if (!tempPath) return
                uni.showLoading({ title: ' 处理中' })
                try {
                    let path = tempPath
                    if (typeof uni.compressImage === 'function') {
                        const compressed = await new Promise((r) => {
                            uni.compressImage({
                                src: tempPath,
                                quality: 80,
                                success: (x) => r(x.tempFilePath),
                                fail: () => r(null)
                            })
                        })
                        if (compressed) path = compressed
                    }
                    this.bg = await saveBackgroundImage(path)
                    uni.showToast({ title: ' 已设置背景', icon: 'success' })
                } finally {
                    uni.hideLoading()
                }
            }
        })
    },
    removeBg() {
        removeBackgroundImage()
        this.bg = ''
        uni.showToast({ title: ' 已移除背景', icon: 'success' })
    }
}
</script>

<style>
    .bg-preview {
        height: 260rpx;
        border-radius: 12rpx;
        margin-bottom: 20rpx;
        display: flex;
        align-items: flex-end;
        justify-content: center;
        overflow: hidden;
```

```
}

.bg-preview-tip {
  font-size: 22rpx;
  color: #fff;
  background: rgba(0, 0, 0, 0.35);
  padding: 6rpx 20rpx;
  border-radius: 20rpx;
  margin-bottom: 16rpx;
}

.bg-actions {
  display: flex;
}

.bg-btn {
  flex: 1;
  height: 76rpx;
  line-height: 76rpx;
  font-size: 26rpx;
  color: var(--c-text-secondary);
  background: var(--c-bg);
  border-radius: 12rpx;
  margin-right: 16rpx;
}

.bg-btn:last-child {
  margin-right: 0;
}

.bg-btn.primary {
  color: #fff;
  background: var(--c-primary);
}
</style>

// ===== FILE: pages/settings/compress.vue =====
<template>
  <view class="ss-page" :class="themeClass">
    <view class="ss-tipbar"> 上下文压缩：长对话自动交给 LLM 压缩为概要，减少后续请求 token 消耗。
    本页设置更改后自动保存。</view>

    <view class="ss-auto">[v] 更改自动保存 </view>

    <view class="ss-card">
      <view class="ss-title"> 自动压缩间隔 </view>
      <view class="ss-row">
        <view class="ss-btns">
          <view
            v-for="o in compressOptions"
```

```

        :key="o.value"
        class="ss-btn first"
        :class="{ on: s.compressInterval === o.value }"
        @tap="setInterval(o.value)"
      >{{ o.label }}</view>
    </view>
    <view class="ss-hint">
      累计新增消息达到设定条数后，自动把上文交给 LLM 压缩成概要；历史消息仍完整保留可
      翻阅，也可在聊天页手动压缩。
    </view>
  </view>
</view>
</template>

<script>
  import { getConversationSettings, saveSettings } from '../utils/chat.js'

  export default {
    data() {
      return {
        s: getConversationSettings(),
        compressOptions: [
          { label: '关闭', value: 0 },
          { label: '20 条', value: 20 },
          { label: '30 条', value: 30 },
          { label: '40 条', value: 40 },
          { label: '60 条', value: 60 },
          { label: '80 条', value: 80 }
        ]
      }
    },
    onShow() {
      this.s = getConversationSettings()
    },
    methods: {
      setInterval(v) {
        if (this.s.compressInterval === v) return
        this.s.compressInterval = v
        saveSettings(this.s) // 更改后自动保存
        uni.showToast({ title: '已保存 (' + (v || '关闭') + ') ', icon: 'none' })
      }
    }
  }
</script>

// ===== FILE: pages/settings/data.vue =====
<template>
  <view class="ss-page" :class="themeClass">
    <view class="ss-tipbar"> 数据管理：清空操作即时生效、无法撤销，请谨慎操作。</view>
  </view>
</template>

```

```
<view class="ss-card">
  <view class="ss-title"> 数据管理 </view>
  <button class="ss-danger-btn" @tap="clearChats"> 清空对话 </button>
  <button class="ss-danger-btn" @tap="clearData"> 清空记忆与对话 </button>
  <view class="ss-hint"> 清空对话仅清空当前会话消息；清空记忆与对话会删除所有记忆与对话记录
    。小程序端需在公众平台配置 request 合法域名；App 端保持联网即可。</view>
</view>
</view>
</template>

<script>
import { clearConversation } from '../utils/chat.js'
import { clearAllData } from '../utils/storage.js'

export default {
  methods: {
    clearChats() {
      uni.showModal({
        title: ' 清空对话',
        content: ' 确定清空对话历史吗?',
        success: (res) => {
          if (res.confirm) {
            clearConversation()
            uni.showToast({ title: ' 已清空', icon: 'success' })
          }
        }
      })
    },
    clearData() {
      uni.showModal({
        title: ' 清空全部数据',
        content: ' 将删除所有记忆与对话记录，且不可恢复。确定继续?',
        success: (res) => {
          if (res.confirm) {
            clearAllData()
            uni.showToast({ title: ' 已清空', icon: 'success' })
          }
        }
      })
    }
  }
}
</script>

<style>
.ss-danger-btn {
  margin-bottom: 20rpx;
  height: 80rpx;
  line-height: 80rpx;
}
```

```

    font-size: 28rpx;
    color: var(--c-danger);
    background: var(--c-card);
    border: 1rpx solid var(--c-danger-light);
    border-radius: 12rpx;
  }
</style>

// ===== FILE: pages/settings/help.vue =====
<template>
  <view class="ss-page" :class="themeClass">
    <view class="ss-tipbar"> 帮助：了解本 App 的各个功能与使用方法。点击条目展开详情。</view>

    <view class="ss-card">
      <view
        v-for="(item, i) in helps"
        :key="item.title"
        class="help-item"
      >
        <view class="help-head" @tap="toggle(i)">
          <text class="help-title">{{ item.title }}</text>
          <text class="help-arrow">{{ open === i ? '-' : '+' }}</text>
        </view>
        <view v-if="open === i" class="help-body">
          <text class="help-text">{{ item.body }}</text>
        </view>
      </view>
    </view>
  </view>
</template>

<script>
  export default {
    data() {
      return {
        open: 0,
        helps: [
          {
            title: ' 开始一次对话',
            body: ' 在聊天页底部输入框打字并发送即开始。首次使用请先到「设置 -> 接口配置」填好接口地址与 API Key，再回聊天页发消息。'
          },
          {
            title: ' 记忆如何运作',
            body: 'AI 会在回复中自动沉淀你的核心事实、约定与近期状态，按重要性分级（L1 核心 / L2 约定 / L3 临时）持续衰减维护。可到「记忆」标签页查看、编辑、删除或手动新建记忆，也可多选批量删除。'
          },
          {
            title: ' 使用表情包',

```

```

        body: ' 到「表情管理」标签页上传图片并逐张命名，聊天时输入框旁的「表情」按钮
可插入 $ 表情名 $。开启「人格配置 -> 聊天表情包」后，AI 也会在你的表情包清单中挑选使用。 '
    },
    {
        title: ' 情景 (Scene) ',
        body: ' 聊天页顶部的情景条会结合当前时间推断「你在做什么」。AI 每次回复都会更
新情景；点击情景条可查看、修改或删除，并从下拉历史中快速复用。 '
    },
    {
        title: ' 拟真聊天',
        body: ' 在「设置 -> 拟真聊天」开启后，AI 会在随机时间主动给你发消息（仅现实时
间模式、且在当前对话生效），每条 <=1 句。可设置主动时段与频率，或自定义倒计时调试。 '
    },
    {
        title: ' 语音阅读 (TTS) ',
        body: ' 在「设置 -> 语音阅读」填入 TTS API Key（默认 Qwen-TTS）并开启后，AI
新回复会自动朗读，表情不朗读、语音不落盘。可用「测试语音接口」验证配置。 '
    },
    {
        title: ' 上下文压缩',
        body: ' 长对话会自动（或手动）把较早上文交给 LLM 压缩成概要，节省后续请求的 t
oken 消耗；原始消息仍完整保留可翻阅。间隔可在「设置 -> 上下文压缩」调整。 '
    },
    {
        title: ' 重新生成与回撤',
        body: 'AI 回复下方有「重新生成」可让 AI 重答；重新生成会撤销上一轮回复及其产
生的情景/记忆变化，保留你的提问。 '
    },
    {
        title: ' 多会话管理',
        body: ' 聊天页头部「历史」可查看、切换、删除会话，或把当前对话复制为副本；「
新对话」开始新会话并复制当前设置。每个会话拥有独立的记忆、情景与设置。 '
    },
    {
        title: ' 导出聊天记录',
        body: ' 在聊天页「历史 -> 导出对话为 .txt」可将当前会话导出为文本：H5 下载、A
pp 保存到文档目录、小程序复制全文。 '
    },
    {
        title: ' 排查问题',
        body: ' 遇到报错或无回复时，到「设置 -> 调试日志」查看请求、响应与错误详情，
可据此确认接口地址/模型名是否正确，判断返回的是否为流式数据。 '
    }
]
},
methods: {
    toggle(i) {
        this.open = this.open === i ? -1 : i
    }
}

```



```
    }
  }
</script>

<style>
  .help-item {
    border-bottom: 1rpx solid var(--c-line);
  }
  .help-item:last-child {
    border-bottom: none;
  }
  .help-head {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 26rpx 6rpx;
  }
  .help-title {
    font-size: 28rpx;
    color: var(--c-text);
    font-weight: 500;
  }
  .help-arrow {
    font-size: 32rpx;
    color: var(--c-text-aid);
    line-height: 1;
  }
  .help-body {
    padding: 0 6rpx 24rpx;
  }
  .help-text {
    font-size: 24rpx;
    color: var(--c-text-secondary);
    line-height: 1.7;
  }
</style>

// ===== FILE: pages/settings/logs.vue =====
<template>
  <view class="ss-page" :class="themeClass">
    <view class="ss-card">
      <view class="ss-title"> 调试日志 </view>
      <view class="log-actions">
        <button class="log-btn primary" @tap="refreshLogs"> 刷新 </button>
        <button class="log-btn" @tap="doClearLogs"> 清空日志 </button>
        <text class="log-count"> 共 {{ logs.length }} 条 </text>
      </view>
      <scroll-view scroll-y class="log-list">
        <view v-if="!logs.length" class="log-empty"> 暂无日志，进行对话或操作后会自动记录 </vi
ew>
```

```
<view
  v-for="l in logs"
  :key="l.id"
  class="log-item"
  :class="{ expanded: expandedId === l.id }"
  @tap="toggleLog(l.id)"
>
  <view class="log-head">
    <text class="log-badge" :class="'t-' + l.type">{{ typeName(l.type) }}</text>
    <text class="log-msg">{{ l.msg }}</text>
    <text class="log-time">{{ fmtTime(l.time) }}</text>
  </view>
  <view v-if="l.detail" class="log-detail">
    <text class="log-detail-text">{{ expandedId === l.id ? l.detail : clipDetail
(1.detail) }}</text>
    <text class="log-toggle">{{ expandedId === l.id ? '收起' : '展开详情' }}</te
xt>
  </view>
</view>
</scroll-view>
</view>
</template>

<script>
import { getLogs, clearLogs as clearDebugLogs } from '../utils/log.js'

const TYPE_NAMES = { req: '请求', res: '响应', err: '错误', info: '信息' }

export default {
  data() {
    return {
      logs: getLogs(),
      expandedId: ''
    }
  },
  onShow() {
    this.logs = getLogs()
  },
  methods: {
    refreshLogs() {
      this.logs = getLogs()
      this.expandedId = ''
    },
    toggleLog(id) {
      this.expandedId = this.expandedId === id ? '' : id
    },
    clipDetail(d) {
      const t = String(d || '')
      return t.length > 200 ? t.slice(0, 200) + '...' : t
    }
  }
}
```

```
    },
    doClearLogs() {
      uni.showModal({
        title: ' 清空日志',
        content: ' 确定清空全部调试日志吗?',
        success: (res) => {
          if (!res.confirm) return
          clearDebugLogs()
          this.logs = []
          this.expandedId = ''
          uni.showToast({ title: ' 已清空日志', icon: 'success' })
        }
      })
    },
    typeName(t) {
      return TYPE_NAMES[t] || t
    },
    fmtTime(iso) {
      if (!iso) return ''
      const d = new Date(iso)
      if (Number.isNaN(d.getTime())) return ''
      const pad = (n) => String(n).padStart(2, '0')
      return `${pad(d.getMonth() + 1)}-${pad(d.getDate())} ${pad(d.getHours())}:${pad(d.getMinutes())}:${pad(d.getSeconds())}`
    }
  }
}
</script>

<style>
  .log-actions {
    display: flex;
    align-items: center;
    margin-bottom: 20rpx;
  }
  .log-btn {
    flex: none;
    width: 200rpx;
    height: 76rpx;
    line-height: 76rpx;
    font-size: 26rpx;
    color: var(--c-text-secondary);
    background: var(--c-bg);
    border-radius: 12rpx;
    margin-right: 16rpx;
  }
  .log-btn.primary {
    color: #fff;
    background: var(--c-primary);
  }
}
```

```
.log-count {
  font-size: 22rpx;
  color: var(--c-text-aid);
}

.log-list {
  height: 1000rpx;
  background: var(--c-bg);
  border-radius: 12rpx;
  padding: 8rpx 20rpx;
  box-sizing: border-box;
}

.log-empty {
  padding: 80rpx 0;
  text-align: center;
  font-size: 24rpx;
  color: var(--c-text-aid);
}

.log-item {
  padding: 16rpx 0;
  border-bottom: 1rpx solid var(--c-line);
}

.log-item:last-child {
  border-bottom: none;
}

.log-item.expanded {
  background: var(--c-card);
  border-radius: 8rpx;
  padding-left: 12rpx;
  padding-right: 12rpx;
}

.log-head {
  display: flex;
  align-items: center;
}

.log-badge {
  flex-shrink: 0;
  font-size: 20rpx;
  padding: 2rpx 12rpx;
  border-radius: 16rpx;
  margin-right: 12rpx;
  color: #fff;
}

.log-badge.t-req { background: var(--c-primary); }
.log-badge.t-res { background: #00b578; }
.log-badge.t-err { background: var(--c-danger); }
.log-badge.t-info { background: #86909c; }

.log-msg {
  flex: 1;
  min-width: 0;
  font-size: 26rpx;
}
```

```
    color: var(--c-text);
    overflow: hidden;
    text-overflow: ellipsis;
    white-space: nowrap;
  }
  .log-time {
    flex-shrink: 0;
    margin-left: 12rpx;
    font-size: 20rpx;
    color: var(--c-text-aid);
  }
  .log-detail {
    margin-top: 8rpx;
    padding: 12rpx 16rpx;
    background: var(--c-bg);
    border-radius: 8rpx;
  }
  .log-detail-text {
    font-size: 22rpx;
    color: var(--c-text-secondary);
    line-height: 1.5;
    word-break: break-all;
    white-space: pre-wrap;
  }
  .log-toggle {
    display: inline-block;
    margin-top: 8rpx;
    font-size: 22rpx;
    color: var(--c-primary);
  }
</style>

// ===== FILE: pages/settings/persona.vue =====
<template>
  <view class="ss-page" :class="themeClass">
    <view class="ss-tipbar"> 人格配置：选择 AI 扮演的角色，设定情景时间与表情包使用。更改后点击底部「保存设置」生效，仅作用于当前对话。</view>

    <view class="ss-card">
      <view class="ss-title"> 情景与表情 </view>
      <view class="ss-row">
        <view class="ss-row-head">
          <text class="ss-label" style="margin-bottom: 0"> 情景时间 </text>
          <view class="ss-btns">
            <view class="ss-btn first" :class="{ on: s.timeMode === 'real' }" @tap="s.timeMode = 'real'"> 现实时间 </view>
            <view class="ss-btn" :class="{ on: s.timeMode === 'virtual' }" @tap="s.timeMode = 'virtual'"> 虚拟时间 </view>
          </view>
        </view>
      </view>
    </view>
  </view>
</template>
```

```

        <view class="ss-hint">
            {{ s.timeMode === 'real' ? ' 会发送当前真实时间，LLM 据此维护现实情景' : ' 不发送
真实时间，情景由 LLM 自由想象（适合角色扮演）' }}
        </view>
    </view>
    <view class="ss-row">
        <view class="ss-row-head">
            <text class="ss-label" style="margin-bottom: 0"> 聊天表情包 </text>
            <view class="ss-btns">
                <view class="ss-btn first" :class="{ on: s.emojiEnabled }" @tap="s.emojiEnab
led = true"> 允许 </view>
                <view class="ss-btn" :class="{ on: !s.emojiEnabled }" @tap="s.emojiEnabled =
false"> 禁用 </view>
            </view>
        </view>
        <view class="ss-hint">
            允许时请求携带表情清单，LLM 回复可穿插使用你的表情包；禁用后 LLM 不会主动使用（
手动插入与渲染不受影响）。
        </view>
    </view>
</view>

<view class="ss-card">
    <view class="ss-title"> 人格选择 </view>
    <view
        v-for="p in personalities"
        :key="p.id"
        class="persona"
        :class="{ active: s.personalityId === p.id }"
        @tap="selectPersona(p.id)"
    >
        <view class="persona-head">
            <view class="persona-main">
                <view class="persona-name">{{ p.name }}</view>
                <view class="persona-desc">{{ p.desc }}</view>
            </view>
            <text class="persona-expand" @tap.stop="togglePersonaPrompt(p.id)">
                {{ expandedPromptId === p.id ? ' 收起提示词' : ' 查看提示词' }}
            </text>
        </view>
        <view v-if="expandedPromptId === p.id" class="persona-prompt">{{ p.prompt }}</view>
    </view>
    <view
        class="persona"
        :class="{ active: s.personalityId === 'custom' }"
        @tap="selectPersona('custom')"
    >
        <view class="persona-head">
            <view class="persona-main">
                <view class="persona-name"> 自定义 </view>

```

```

        <view class="persona-desc"> 参考下方示例或预设人格的提示词，手写你的专属设定 <
    /view>

        </view>
        <text class="persona-expand" @tap.stop="togglePersonaPrompt('__sample__')">
            {{ expandedPromptId === '__sample__' ? '收起示例' : '查看完整示例' }}
        </text>
    </view>
    <view v-if="expandedPromptId === '__sample__'" class="persona-prompt">
        <text>{{ samplePrompt }}</text>
        <view class="prompt-fill" @tap.stop="fillSample"> 一键填入到下方编辑框 </view>
    </view>
</view>
<textarea
    v-if="s.personalityId === 'custom'"
    class="custom-box"
    v-model="s.customPrompt"
    maxlength="5000"
    placeholder=" 在这里输入完整的人格设定提示词..."
/>
    <view v-if="s.personalityId === 'custom'" class="char-count">{{ s.customPrompt.length }}
/ 5000</view>
</view>

    <view class="ss-save" @tap="save"> 保存设置 </view>
</view>
</template>

<script>
import { getConversationSettings, saveSettings } from '../utils/chat.js'
import { PERSONALITIES, CUSTOM_PROMPT_SAMPLE } from '../utils/prompts.js'
import { addLog } from '../utils/log.js'

export default {
    data() {
        return {
            s: getConversationSettings(),
            personalities: PERSONALITIES,
            expandedPromptId: '',
            samplePrompt: CUSTOM_PROMPT_SAMPLE
        }
    },
    onShow() {
        this.s = getConversationSettings()
    },
    methods: {
        selectPersona(id) {
            this.s.personalityId = id
        },
        togglePersonaPrompt(id) {
            this.expandedPromptId = this.expandedPromptId === id ? '' : id
        }
    }
}

```

```
    },
    fillSample() {
      this.s.customPrompt = CUSTOM_PROMPT_SAMPLE
      uni.showToast({ title: ' 已填入示例，可在此基础上修改', icon: 'none' })
    },
    save() {
      saveSettings(this.s)
      addLog('info', ' 保存设置', `人格 =${this.s.personalityId} . 时间模式 =${this.s.timeMode} === 'virtual' ? ' 虚拟' : ' 现实' . 表情包 =${this.s.emojiEnabled ? ' 允许' : ' 禁用'}`)
      uni.showToast({ title: ' 已保存', icon: 'success' })
    }
  }
}
</script>
```

```
<style>
.persona {
  padding: 20rpx 24rpx;
  border-radius: 12rpx;
  background: var(--c-bg);
  margin-bottom: 16rpx;
  border: 2rpx solid transparent;
}
.persona.active {
  border-color: var(--c-primary);
  background: var(--c-primary-light);
}
.persona-name {
  font-size: 28rpx;
  color: var(--c-text);
  font-weight: 500;
}
.persona-desc {
  font-size: 22rpx;
  color: var(--c-text-aid);
  margin-top: 6rpx;
}
.persona-head {
  display: flex;
  align-items: flex-start;
}
.persona-main {
  flex: 1;
  min-width: 0;
}
.persona-expand {
  flex-shrink: 0;
  margin-left: 16rpx;
  padding-top: 4rpx;
  font-size: 22rpx;
}
```



```
        color: var(--c-primary);
    }
    .persona-prompt {
        margin-top: 16rpx;
        padding: 20rpx;
        background: var(--c-bg);
        border-radius: 12rpx;
        border: 1rpx solid var(--c-line);
        font-size: 24rpx;
        color: var(--c-text-secondary);
        line-height: 1.7;
        white-space: pre-wrap;
        word-break: break-word;
    }
    .prompt-fill {
        margin-top: 16rpx;
        display: inline-block;
        font-size: 24rpx;
        color: var(--c-primary);
        border: 1rpx solid var(--c-primary);
        border-radius: 24rpx;
        padding: 8rpx 24rpx;
    }
    .custom-box {
        width: 100%;
        height: 800rpx;
        background: var(--c-bg);
        border-radius: 12rpx;
        padding: 20rpx;
        box-sizing: border-box;
        font-size: 26rpx;
        color: var(--c-text);
        line-height: 1.6;
    }
    .char-count {
        margin-top: 10rpx;
        text-align: right;
        font-size: 22rpx;
        color: var(--c-text-aid);
    }
}
</style>

// ===== FILE: pages/settings/proactive.vue =====
<template>
    <view class="ss-page" :class="themeClass">
        <view class="ss-tipbar"> 拟真聊天：让 AI 更像真人、在随机时间主动给你发消息。更改后点击底部「
        保存设置」生效。仅「现实时间」模式生效。若要后台接收通知，请在手机设置打开本应用的通知权限 </view>

        <view class="ss-card">
            <view class="ss-title"> 拟真聊天 </view>
```

```

    <view class="ss-row">
      <view class="ss-row-head">
        <text class="ss-label" style="margin-bottom: 0"> 开关 </text>
        <view class="ss-btns">
          <view class="ss-btn first" :class="{ on: s.proactiveEnabled }" @tap="s.proac
tiveEnabled = true"> 开启 </view>
          <view class="ss-btn" :class="{ on: !s.proactiveEnabled }" @tap="s.proactiveE
nabled = false"> 关闭 </view>
        </view>
      </view>
      <view class="ss-hint"> 开启后 AI 会在随机时间主动发消息（每条 <=1 句、连续表情 <=2）
；开启本对话的所有回复都按该短消息规则约束。</view>
    </view>
    <view class="ss-row">
      <view class="ss-row-head">
        <text class="ss-label" style="margin-bottom: 0"> 主动消息时段 </text>
        <view class="proactive-window">
          <picker
            mode="time"
            :value="minToTime(s.proactiveStartMin)"
            @change="(e) => s.proactiveStartMin = timeToMin(e.detail.value)"
          >
            <view class="pick-btn">{{ minToTime(s.proactiveStartMin) }}</view>
          </picker>
          <text class="window-sep">--</text>
          <picker
            mode="time"
            :value="minToTime(s.proactiveEndMin)"
            @change="(e) => s.proactiveEndMin = timeToMin(e.detail.value)"
          >
            <view class="pick-btn">{{ minToTime(s.proactiveEndMin) }}</view>
          </picker>
        </view>
      </view>
      <view class="ss-hint"> 仅在此时间段内 AI 会主动发消息（精确到分钟），其余时间静默。</
view>
    </view>
    <view class="ss-row">
      <view class="ss-row-head">
        <text class="ss-label" style="margin-bottom: 0"> 主动消息频率 </text>
        <view class="ss-btns">
          <view class="ss-btn first" :class="{ on: s.proactiveCustomSeconds <= 0 && s.
proactiveLevel === 'low' }" @tap="setLevel('low')"> 低频 </view>
          <view class="ss-btn" :class="{ on: s.proactiveCustomSeconds <= 0 && s.proact
iveLevel === 'medium' }" @tap="setLevel('medium')"> 中频 </view>
          <view class="ss-btn" :class="{ on: s.proactiveCustomSeconds <= 0 && s.proact
iveLevel === 'high' }" @tap="setLevel('high')"> 高频 </view>
          <view class="ss-btn" :class="{ on: s.proactiveCustomSeconds > 0 }" @tap="s.p
roactiveCustomSeconds = s.proactiveCustomSeconds > 0 ? s.proactiveCustomSeconds : 30"> 自定义 </view>
        </view>
      </view>
    </view>
  </view>

```

```

    </view>
    <view class="ss-hint"> 低频 45~120 分钟 | 中频 15~45 分钟 | 高频 5~15 分钟。填秒数则固
定按该倒计时触发（便于调试）。</view>
    <view class="ss-field">
      <text class="ss-label"> 自定义倒计时（秒）</text>
      <input class="ss-input" type="number" v-model="s.proactiveCustomSeconds" placeho
lder="0 = 使用上方档位" />
    </view>
  </view>
  <view class="ss-row">
    <view class="ss-row-head">
      <text class="ss-label" style="margin-bottom: 0"> 调试 </text>
      <view class="ss-btns">
        <view class="ss-btn test" :class="{ disabled: debugging }" @tap="debugProact
ive"> 立即发送一条主动消息 </view>
      </view>
    </view>
    <view class="ss-hint"> 忽略时段/间隔，按当前表单配置立即请求模型返回一条主动消息（发
送前自动保存设置）。</view>
    <view class="countdown">{{ countdown }}</view>
  </view>
</view>

  <view class="ss-save" @tap="save"> 保存设置 </view>
</view>
</template>

<script>
  import { getConversationSettings, saveSettings } from '../utils/chat.js'
  import { debugProactiveMessage, rearmProactive, getProactiveCountdown } from '../utils/chat-p
roactive.js'
  import { requestNotifyPermission } from '../utils/notify.js'
  import { addLog } from '../utils/log.js'

  export default {
    data() {
      return {
        s: getConversationSettings(),
        debugging: false,
        countdown: '',
        _countdownTimer: null
      }
    },
    onShow() {
      this.s = getConversationSettings()
      this.updateCountdown()
      if (this._countdownTimer) clearInterval(this._countdownTimer)
      this._countdownTimer = setInterval(() => this.updateCountdown(), 1000)
    },
    onHide() {

```

```

    if (this._countdownTimer) {
      clearInterval(this._countdownTimer)
      this._countdownTimer = null
    }
  },
  methods: {
    setLevel(lv) {
      this.s.proactiveCustomSeconds = 0
      this.s.proactiveLevel = lv
    },
    // 分钟 -> "HH:MM" (钳制 0-1439, 非法归 0)
    minToTime(min) {
      const m = Math.max(0, Math.min(1439, parseInt(min, 10) || 0))
      const h = Math.floor(m / 60)
      const mm = m % 60
      return (h < 10 ? '0' + h : '' + h) + ':' + (mm < 10 ? '0' + mm : '' + mm)
    },
    // "HH:MM" -> 当天第几分钟 (0-1439)
    timeToMin(str) {
      const p = String(str || '').split(':')
      const h = parseInt(p[0], 10) || 0
      const mm = parseInt(p[1], 10) || 0
      return Math.max(0, Math.min(1439, h * 60 + mm))
    },
    updateCountdown() {
      const ms = getProactiveCountdown()
      if (ms === null) {
        const s = this.s || getConversationSettings()
        if (!s.proactiveEnabled) this.countdown = '未开启拟真聊天'
        else if (s.timeMode !== 'real') this.countdown = '仅现实时间模式生效'
        else this.countdown = '等待调度'
      } else if (ms === 0) {
        this.countdown = '即将发送主动消息...'
      } else {
        const total = Math.ceil(ms / 1000)
        const h = Math.floor(total / 3600)
        const m = Math.floor((total % 3600) / 60)
        const sec = total % 60
        this.countdown = h > 0
          ? `下次主动消息: ${h} 小时 ${m} 分`
          : `下次主动消息: ${m} 分 ${String(sec).padStart(2, '0')} 秒`
      }
    },
    async debugProactive() {
      if (this.debugging) return
      if (!this.s.apiKey.trim()) {
        uni.showToast({ title: '请填写 API Key', icon: 'none' })
        return
      }
    },
    saveSettings(this.s)
  }
}

```

```

        rearmProactive()
        this.debugging = true
        try {
            const r = await debugProactiveMessage()
            if (r && r.lines && r.lines.length) {
                uni.showToast({ title: ' 已发送 ' + r.lines.length + ' 条', icon: 'success' })
            } else {
                uni.showToast({ title: ' 发送被跳过 (请求进行中或未配置 API)', icon: 'none' })
            }
        } catch (e) {
            uni.showToast({ title: (e && e.message) || ' 发送失败', icon: 'none' })
        } finally {
            this.debugging = false
        }
    },
    save() {
        if (!this.s.apiKey.trim()) {
            uni.showToast({ title: ' 请填写 API Key', icon: 'none' })
            return
        }
        saveSettings(this.s)
        rearmProactive()
        // 开启拟真时申请系统通知权限 (后台收到主动消息时弹通知)
        if (this.s.proactiveEnabled) requestNotifyPermission()
        addLog(
            'info',
            ' 保存设置',
            `拟真 =${this.s.proactiveEnabled ? ' 开 (时段' + this.minToTime(this.s.proactiveStartMin) + '-' + this.minToTime(this.s.proactiveEndMin) + ', ' + (this.s.proactiveCustomSeconds > 0 ? ' 自定义' + this.s.proactiveCustomSeconds + ' 秒' : this.s.proactiveLevel + ' 档') + ')': '关'}`
        )
        uni.showToast({ title: ' 已保存', icon: 'success' })
    }
}
}
</script>

<style>
.proactive-window {
    display: flex;
    align-items: center;
}
.pick-btn {
    padding: 8rpx 24rpx;
    border-radius: 12rpx;
    background: var(--c-card);
    border: 1rpx solid var(--c-line);
    font-size: 26rpx;

```

```
        color: var(--c-text);
    }
    .window-sep {
        margin: 0 16rpx;
        color: var(--c-text-aid);
        font-size: 26rpx;
    }
    .countdown {
        margin-top: 12rpx;
        color: var(--c-primary);
        font-weight: 600;
        font-size: 24rpx;
    }
}
</style>

// ===== FILE: pages/settings/settings.vue =====
<template>
    <view class="page" :class="themeClass">
        <view class="hero">
            <view class="hero-title"> 设置 </view>
            <view class="hero-sub"> 每项设置进入对应页面调整，更改后记得保存 </view>
        </view>

        <view class="group" v-for="g in groups" :key="g.name">
            <view class="group-name">{{ g.name }}</view>
            <view class="group-card">
                <view
                    v-for="item in g.items"
                    :key="item.page"
                    class="entry"
                    @tap="go(item.page)"
                >
                    <view class="entry-main">
                        <view class="entry-name">
                            {{ item.name }}
                            <text v-if="item.required && needConfig" class="req">*</text>
                            <text v-if="item.tag" class="tag">{{ item.tag }}</text>
                        </view>
                        <view class="entry-desc">{{ item.desc }}</view>
                    </view>
                    <text class="entry-arrow">></text>
                </view>
            </view>
        </view>

        <!-- 自绘底部导航栏（替代原生 tabBar，随主题深浅色切换） -->
        <app-tab-bar :active="2" />
    </view>
</template>
```

```

<script>
  import { getConversationSettings } from '../utils/chat.js'
  import AppTabBar from '../components/app-tab-bar/app-tab-bar.vue'

  export default {
    components: { AppTabBar },
    data() {
      return {
        needConfig: false, // 接口配置缺失 (baseUrl/apiKey 未填) 时置 true
        groups: [
          {
            name: ' 常规',
            items: [
              { page: '/pages/settings/appearance', name: ' 界面外观', desc: ' 深色模式、聊天气泡不透明度与聊天背景', tag: ' 保存' },
              { page: '/pages/settings/api', name: ' 接口配置', desc: ' LLM 接口地址 / Key / 模型 / 预设与配置教学', required: true, tag: ' 保存' },
              { page: '/pages/settings/persona', name: ' 人格配置', desc: ' 人格选择、情景时间、自定义设定', tag: ' 保存' },
              { page: '/pages/settings/proactive', name: ' 拟真聊天', desc: ' 模拟真人主动发消息的时段与频率', tag: ' 保存' },
              { page: '/pages/settings/tts', name: ' 语音阅读', desc: ' 回复自动朗读的 TTS 接口与音色', tag: ' 保存' }
            ]
          },
          {
            name: ' 对话与性能',
            items: [
              { page: '/pages/settings/compress', name: ' 上下文压缩', desc: ' 长对话自动压缩为概要, 节省 token', tag: ' 自动保存' }
            ]
          },
          {
            name: ' 数据与调试',
            items: [
              { page: '/pages/settings/data', name: ' 数据管理', desc: ' 清空对话 / 清空记忆与全部数据' },
              { page: '/pages/settings/logs', name: ' 调试日志', desc: ' 查看请求、响应、错误与操作记录' }
            ]
          },
          {
            name: ' 关于',
            items: [
              { page: '/pages/settings/help', name: ' 帮助', desc: ' 各功能介绍与使用方法' },
              { page: '/pages/settings/about', name: ' 关于', desc: ' App 版本与技术信息' }
            ]
          }
        ]
      }
    }
  }

```

```
    ]
  },
  onShow() {
    // 接口配置为必要项：任一缺失（地址或 Key 为空）即显示红色 * 提示
    const s = getConversationSettings()
    this.needConfig = !(s.baseUrl && s.baseUrl.trim() && s.apiKey && s.apiKey.trim())
  },
  methods: {
    go(page) {
      uni.navigateTo({ url: page })
    }
  }
}
</script>

<style>
  .page {
    padding: 20rpx 30rpx calc(var(--ctab-h, 100rpx) + env(safe-area-inset-bottom) + 40rpx);
    background: var(--c-bg);
    min-height: 100vh;
    box-sizing: border-box;
  }

  .hero {
    padding: 20rpx 6rpx 28rpx;
  }

  .hero-title {
    font-size: 44rpx;
    font-weight: 700;
    color: var(--c-text);
  }

  .hero-sub {
    margin-top: 8rpx;
    font-size: 24rpx;
    color: var(--c-text-aid);
  }

  .group {
    margin-bottom: 24rpx;
  }

  .group-name {
    font-size: 24rpx;
    color: var(--c-text-aid);
    margin: 0 6rpx 12rpx;
  }

  .group-card {
    background: var(--c-card);
    border-radius: var(--c-radius-lg);
    box-shadow: var(--c-shadow-card);
  }
</style>
```



```
        overflow: hidden;
        border: 1rpx solid transparent;
    }

    .entry {
        display: flex;
        align-items: center;
        padding: 26rpx 28rpx;
        border-bottom: 1rpx solid var(--c-line);
    }

    .entry:last-child {
        border-bottom: none;
    }

    .entry-name {
        font-size: 28rpx;
        color: var(--c-text);
        font-weight: 500;
        display: flex;
        align-items: center;
    }

    .entry-name .req {
        color: var(--c-danger);
        margin-left: 6rpx;
        font-weight: 700;
    }

    .entry-name .tag {
        margin-left: 12rpx;
        font-size: 20rpx;
        color: var(--c-text-aid);
        font-weight: 400;
        border: 1rpx solid var(--c-line);
        border-radius: 20rpx;
        padding: 2rpx 12rpx;
    }

    .entry-desc {
        margin-top: 6rpx;
        font-size: 22rpx;
        color: var(--c-text-aid);
        line-height: 1.5;
    }

    .entry-arrow {
        margin-left: 16rpx;
        font-size: 40rpx;
        color: var(--c-text-aid);
        line-height: 1;
    }
}

</style>

// ===== FILE: pages/settings/tts.vue =====

<template>
```

```
<view class="ss-page" :class="themeClass">
  <view class="ss-tipbar"> 语音阅读：LLM 新返回的消息在渲染时自动合成语音朗读，表情不朗读、语音
  不落盘。更改后点击底部「保存设置」生效。</view>

  <view class="ss-card">
    <view class="ss-title"> 语音阅读 </view>
    <view class="ss-row">
      <view class="ss-row-head">
        <text class="ss-label" style="margin-bottom: 0"> 开关 </text>
        <view class="ss-btns">
          <view class="ss-btn first" :class="{ on: s.ttsEnabled }" @tap="s.ttsEnabled
= true"> 开启 </view>
          <view class="ss-btn" :class="{ on: !s.ttsEnabled }" @tap="s.ttsEnabled = fal
se"> 关闭 </view>
        </view>
      </view>
      <view class="ss-hint"> 默认以 Qwen-TTS 为例。开启后需填下面的 API Key 才会朗读。</vie
w>

    </view>
    <view class="ss-field">
      <text class="ss-label">TTS API Key</text>
      <input class="ss-input" v-model="s.ttsApiKey" password placeholder="sk-..." />
    </view>
    <view class="ss-field">
      <text class="ss-label">TTS 模型 </text>
      <input class="ss-input" v-model="s.ttsModel" placeholder="qwen3-tts-flash" />
    </view>
    <view class="ss-field">
      <text class="ss-label"> 音色 </text>
      <view class="voice-row">
        <input class="ss-input voice-input" v-model="s.ttsVoice" placeholder="Cherry" />
        <picker mode="selector" :range="ttsVoiceNames" @change="onTtsVoiceChange">
          <view class="voice-pick-btn"> 从列表选择 </view>
        </picker>
      </view>
      <view v-if="ttsVoiceHint" class="ss-hint">{{ ttsVoiceHint }}</view>
      <view class="ss-hint"> 可手动输入官方列表外的自定义音色；完整音色列表参考官方文档。</
view>

    </view>
    <view class="ss-field">
      <view class="tts-test-btn" @tap="doTestTts"> 测试语音接口 </view>
      <view class="ss-hint"> 按上方当前配置（Key/模型/音色）发送一小段测试文本并尝试播放。<
/view>

    </view>
  </view>

  <view class="ss-save" @tap="save"> 保存设置 </view>
</view>
</template>
```

```
<script>
import { getConversationSettings, saveSettings } from '../utils/chat.js'
import { TTS_VOICE_NAMES, ttsVoiceDesc, testTts as runTtsTest } from '../utils/tts.js'
import { addLog } from '../utils/log.js'

export default {
  data() {
    return {
      s: getConversationSettings(),
      ttsVoiceNames: TTS_VOICE_NAMES
    }
  },
  computed: {
    ttsVoiceHint() {
      const desc = ttsVoiceDesc(this.s.ttsVoice)
      return desc ? '[' + this.s.ttsVoice + ']' + desc : ''
    }
  },
  onShow() {
    this.s = getConversationSettings()
  },
  methods: {
    onTtsVoiceChange(e) {
      this.s.ttsVoice = this.ttsVoiceNames[e.detail.value]
    },
    async doTestTts() {
      if (!this.s.ttsApiKey.trim()) {
        uni.showToast({ title: ' 请先填写 TTS API Key', icon: 'none' })
        return
      }
      uni.showLoading({ title: ' 测试中' })
      let res = { ok: false, message: ' 测试失败' }
      try {
        res = await runTtsTest({ apiKey: this.s.ttsApiKey, model: this.s.ttsModel, voice
: this.s.ttsVoice })
      } catch (e) {
        res = { ok: false, message: (e && e.message) || ' 测试失败' }
      } finally {
        uni.hideLoading()
      }
      uni.showToast({ title: res.message, icon: res.ok ? 'success' : 'none' })
    },
    save() {
      saveSettings(this.s)
      addLog('info', ' 保存设置', `语音阅读 =${this.s.ttsEnabled ? ' 开 (' + (this.s.ttsModel
|| '') + '/' + (this.s.ttsVoice || '') + ')' : ' 关'}`)
      uni.showToast({ title: ' 已保存', icon: 'success' })
    }
  }
}
```

```
</script>
```

```
<style>
```

```
.voice-row {
  display: flex;
  align-items: center;
}

.voice-input {
  flex: 1;
  min-width: 0;
  margin-right: 16rpx;
}

.voice-pick-btn {
  flex-shrink: 0;
  height: 80rpx;
  line-height: 80rpx;
  padding: 0 28rpx;
  font-size: 26rpx;
  color: var(--c-primary);
  background: var(--c-primary-light);
  border-radius: 12rpx;
}

.tts-test-btn {
  height: 80rpx;
  line-height: 80rpx;
  text-align: center;
  font-size: 28rpx;
  color: #fff;
  background: var(--c-brand-gradient);
  border-radius: var(--c-radius-md);
  box-shadow: 0 4rpx 12rpx rgba(91, 124, 250, 0.3);
}
```

```
</style>
```

```
// ===== FILE: uni.promisify.adaptor.js =====
```

```
uni.addInterceptor({
  returnValue (res) {
    if (!!res && (typeof res === "object" || typeof res === "function") && typeof res.then === "function")) {
      return res;
    }
    return new Promise((resolve, reject) => {
      res.then((res) => {
        if (!res) return resolve(res)
        return res[0] ? reject(res[0]) : resolve(res[1])
      });
    });
  },
});
```